СОДЕРЖАНИЕ

ВВЕДЕНИЕ 6
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 8
1.1. Системы коммуникации в реальном времени 8
1.1.1. Классификация режимов коммуникации 8
1.1.2. Обзор существующих систем 9
1.1.3. Метрики качества передачи данных в системах реального времени 13
1.1.4. Требования к клиентским устройствам в системах реального времени 15
1.2. Протоколы передачи данных в реальном времени 16
1.2.1. WebSocket и Socket.IO 16
1.2.2. Message Queue Telemetry Transport 17
1.2.3. SIP 19
1.2.4. RTP, SRTP и RTCP 21
1.2.5. WebRTC 22
1.2.6. Современные протоколы медиадоставки 29
1.3. Архитектура многопользовательских коммуникационных систем 31
1.3.1. Полносвязная P2P-сеть 31
1.3.2. Клиент-серверные архитектуры 32
1.3.3. Гибридные топологии 33
1.3.4. Сравнительный анализ архитектур 34
1.4. Ячеистые топологии и распределённое управление 36
1.4.1. Сотовый принцип в телекоммуникациях 36
1.4.2. Алгоритмы выбора лидера в распределённых системах 36
1.4.3. SFU как инструмент межсегментной маршрутизации 37
1.4.4. Механизм pipe между роутерами 37
Выводы по главе 38
ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ 40
2.1. Требования и концепция системы 40
2.1.1. Функциональные и нефункциональные требования 40

2.1.2. Общая архитектура системы 41
2.1.3. Математическое обоснование эффективности гибридной архитектуры 43
2.2. Проектирование серверной подсистемы 51
2.2.1. Модель данных 51
2.2.2. Алгоритм выбора передатчика 52
2.3. Проектирование коммуникационного взаимодействия 54
2.3.1. Внутриячеечное P2P-взаимодействие 54
2.3.2. Передача текстовых сообщений 55
2.3.3. Межъячеечная передача медиапотоков 55
2.4. Проектирование клиентского приложения 56
2.4.1. Архитектура клиентского приложения 56
2.4.2. Организация пользовательского интерфейса 58
Выводы по главе 58
ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ 61
3.1. Выбор технологического стека 61
3.2. Реализация ключевых компонентов 62
3.2.1. Реализация серверной части 62
3.2.2. Реализация P2P-коммуникации 64
3.2.3. Реализация SFU-интеграции 65
3.2.4. Реализация оценки устройства 67
3.3. Реализация пользовательского интерфейса 68
3.3.1. HomeScreen 68
3.3.2. RoomScreen 68
3.4. Тестирование системы 69
3.4.1. Функциональное тестирование 69
3.4.2. Аналитическое нагрузочное моделирование 69
Выводы по главе 73
ЗАКЛЮЧЕНИЕ 75
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 77
ПРИЛОЖЕНИЯ 82

Приложение А. Рисунки 82
Приложение Б. Таблицы 85
Приложение В. Фрагменты кода реализации 89
В.1. Реализация серверной части 89
В.2. Реализация P2P-коммуникации 94
В.3. Реализация SFU-интеграции 98
В.4. Реализация оценки устройства 102
В.5. Пользовательский интерфейс 103

ВВЕДЕНИЕ

Потребность в системах удалённой коммуникации значительно возросла: переход организаций на дистанционный и гибридный формат работы сформировал устойчивый спрос на средства взаимодействия независимо от местонахождения участников. Такие системы должны функционировать на устройствах с различными техническими характеристиками, в сетях с нестабильным качеством соединения и обеспечивать приемлемое качество передачи голоса и видео.

Актуальность темы обусловлена растущей потребностью в средствах коммуникации, способных эффективно работать в условиях разнородных клиентских устройств и нестабильных сетевых соединений. Существующие платформы используют преимущественно централизованную архитектуру, при которой медиапотоки проходят через серверы поставщика, — это требует значительных инфраструктурных ресурсов и создаёт единую точку отказа. Децентрализованные решения снижают зависимость от серверной инфраструктуры и уменьшают задержку, однако с ростом числа участников нагрузка на устройства быстро возрастает и становится критичной для мобильных платформ. Предложенное решение объединяет преимущества обоих подходов: прямые соединения применяются внутри небольших групп, а серверная маршрутизация подключается только при их расширении, что повышает масштабируемость системы и снижает нагрузку как на клиентские устройства, так и на серверную инфраструктуру.

Практическая значимость работы заключается в создании воспроизводимого прототипа и математической модели распределения нагрузки, применимых при разработке коммуникационной инфраструктуры, независимой от сторонних поставщиков.

Целью исследования является разработка системы текстовой, голосовой и видео коммуникации в реальном времени с гибридной архитектурой,

сочетающей прямую передачу медиаданных внутри динамически формируемых малых групп участников и серверную маршрутизацию между ними.

Для достижения поставленной цели были сформулированы следующие задачи:

1. Провести анализ архитектур и протоколов систем коммуникации реального времени.
2. Спроектировать гибридную ячеистую архитектуру с алгоритмом автоматического выбора узла-передатчика.
3. Реализовать серверное и многоплатформенное клиентское приложения.
4. Провести тестирование системы в ключевых сценариях работы.

Объект исследования – процесс многоточечной передачи медиаданных в реальном времени в распределённых системах с разнородными клиентскими устройствами. Предмет исследования – гибридная ячеистая архитектура системы коммуникации, сочетающая прямые соединения внутри малых групп с серверной маршрутизацией между ними и централизованным алгоритмом выбора узла-передатчика.

В работе применяются сравнительный анализ архитектур и протоколов передачи данных реального времени, математическое моделирование на основе теории графов и сетевых метрик, проектирование с применением унифицированного языка моделирования, а также функциональное тестирование ключевых сценариев взаимодействия.

Работа состоит из введения, трёх глав, заключения, списка использованных источников и приложения. В первой главе проводится анализ предметной области, рассматриваются архитектуры многопользовательских систем, включая ячеистые топологии и алгоритмы выбора управляющего узла; во второй строится математическая модель гибридной архитектуры и выполняется проектирование подсистем; в третьей описываются реализация компонентов и результаты тестирования.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Системы коммуникации в реальном времени

1.1.1. Классификация режимов коммуникации

Проектирование систем реального времени определяется как техническими ограничениями сети, так и особенностями человеческого восприятия. Допустимая задержка передачи данных зависит не только от характеристик оборудования и каналов связи, но и от порогов восприятия, при превышении которых взаимодействие перестаёт восприниматься пользователем как естественное. Архитектура коммуникационных протоколов вырастает именно из этого зазора: допустимая сетевая задержка определяется не столько возможностями оборудования, сколько порогами, за которыми взаимодействие перестаёт восприниматься как естественное.

Наиболее критичным параметром для голоса оказывается односторонняя задержка. Документы ITU-T устанавливают целевые значения: предпочтительная задержка для разговора – менее 150 мс, предельно допустимая – 400мс [1]. Столь жёсткие границы обусловлены двумя механизмами деградации. Во-первых, задержка порядка десятков миллисекунд начинает создавать эхо, разрушающее разборчивость речи [1]. Во-вторых, при нарастании задержки до нескольких сотен миллисекунд нарушается динамика разговора: паузы перестают совпадать с ожидаемыми, собеседники начинают перебивать друг друга, естественный ритм диалога теряется [1]. Второй эффект, связанный с когнитивным восприятием темпа беседы, часто оказывается более разрушительным, чем акустические искажения.

Видеосвязь добавляет дополнительные сложности к передаче данных. Спецификация WebRTC определяет способность конечных точек отправлять и принимать интерактивное видео, источником которого может служить камера, запись экрана, сохранённый файл или иной источник [2]. При этом для режима «видеотелефона» – полнодуплексной передачи аудио и видео – к тем же

требованиям по задержке добавляется необходимость синхронизации: расхождение между аудио и видеопотоком не должно превышать 80 мс [1]. Эта величина жёстче предельной задержки для голоса, поскольку рассогласование артикуляции и звука воспринимается мозгом практически мгновенно.

Нормативы информационных потерь также сформулированы в терминах восприятия. Для разговора пороговым считается коэффициент потерь пакетов на уровне 3% [1]. Ухо относительно терпимо к искажениям речевого сигнала, однако при использовании низкобитрейтных кодеков потери пакетов ведут к катастрофическому падению качества [1]. Таким образом, компрессия голоса, снижая требования к полосе пропускания, одновременно повышает уязвимость к потерям.

Текстовый режим, с формальной точки зрения, предъявляет наименьшие требования к задержке и пропускной способности – и именно поэтому он редко рассматривается инженерами в категории «реального времени». Граница между текстовым и голосовым общением пролегает не в технической, а в плоскости восприятия: от текста не ожидают быстрого отклика, тогда как для голоса такое ожидание встроено в пользовательский опыт.

Стандарты закрепляют эти требования на уровне обязательных реализаций: RFC 8825 предписывает, что конечные точки WebRTC, поддерживающие аудио и/или видео, должны реализовывать кодеки и профили, перечисленные в RFC 7874 и RFC 7742 [3].

Таким образом, требования к системам реального времени определяются не только характеристиками сети, но и психофизиологическими ограничениями человека, что оказывает прямое влияние на проектирование протоколов и архитектур.

1.1.2. Обзор существующих систем

Zoom [4]. Компания долгое время не раскрывала архитектурных деталей, что породило множество спекуляций. Сегодня, однако, консенсус практически сложился: Zoom использует SFU-архитектуру, а не MCU. Исследователи,

анализировавшие поведение системы, отмечают, что распределённые серверы Zoom работают именно как Selective Forwarding Units, пересылая медиапотоки без перекодирования. Это объясняет, почему платформа хорошо масштабируется: SFU не выполняет ресурсоёмкого микширования на серверной стороне.

Впрочем, оговорка необходима. Zoom не построен на чистом WebRTC – его клиент использует собственный стек протоколов, и лишь в браузерной версии задействуется WebRTC-шлюз. Серверная инфраструктура распределена глобально, и клиент подключается к ближайшему дата-центру, а не к единому центральному узлу. Это ключевой момент: масштабирование достигается не столько архитектурой отдельного сервера, сколько географической распределённостью SFU.

Интересно, что Zoom начинал с относительно простой схемы, но по мере роста (взрывной рост пришёлся на 2020 год) был вынужден решать проблемы, с которыми сталкивается любая крупная SFU-система: как маршрутизировать потоки между дата-центрами, когда участники одной конференции физически находятся на разных континентах. Решение – каскадное соединение SFU – превращает каждую конференцию в распределённую топологию, где медиа идёт через несколько серверов, но для клиента это остаётся прозрачным.

Google Meet [5] пошёл другим путём – и в чём-то более радикальным. Платформа построена вокруг концепции виртуальных медиапотоков (virtual media streams), генерируемых SFU для агрегации и распределения контента от множества участников. В отличие от прямых P2P-соединений, создающих сложную mesh-структуру в больших конференциях, виртуальные потоки упрощают топологию: SFU принимает индивидуальные потоки от каждого участника и решает, кому что направить.

Google Meet примечателен тем, как он использует simulcast. SFU получает от клиента несколько пространственных слоёв (разные разрешения) и самостоятельно определяет, какой слой переслать конкретному получателю, основываясь на доступной полосе. Более того, клиент может передавать серверу

метаданные о целевых битрейтах, разрешениях и частоте кадров для каждого слоя – это позволяет SFU принимать решения о пересылке не на основе эвристик, а на основе явных подсказок от отправителя.

Ещё одна особенность Google Meet – отказ от ICE-серверов. Платформа не конфигурирует STUN/TURN для клиентов, используя собственную инфраструктуру для обеспечения связности. Детали этого решения закрыты, но результат налицо: клиентский код проще, а настройка соединения быстрее.

Discord [6] интересен тем, что изначально проектировался не как видеоконференция, а как голосовой чат для игровых сообществ. Отсюда – специфические архитектурные решения. Голосовой сервер Discord состоит из двух компонентов: сигнального и медиа-ретранслятора, называемого SFU. Сигнальный компонент полностью контролирует SFU, управляя генерацией идентификаторов потоков, ключами шифрования и индикацией говорящего.

С точки зрения протоколов Discord использует WebRTC, но с собственной системой шифрования. Поверх стандартного DTLS-SRTP накладывается дополнительный слой: медиа-отправители шифруют закодированные аудио- или видео-фреймы симметричным ключом, извлекаемым из группы MLS (Message Layer Security). Это обеспечивает end-to-end шифрование даже при прохождении трафика через SFU – технически сервер остаётся ненадёжным элементом, не имеющим доступа к содержимому.

Интересной деталью является то, что Discord отображает всех участников медиасессии до тех пор, пока они подключены к голосовому шлюзу и могут получать медиа от SFU. Это создаёт иллюзию постоянного присутствия, что важно для социального опыта в игровых сообществах.

Microsoft Teams [7]. Платформа выросла из Skype for Business и унаследовала поддержку разнородных клиентов и протоколов. Современная архитектура Teams использует SlimCore – оптимизированный стек, который может работать совместно с WebRTC Redirector Service. Этот Redirector позволяет клиенту переключаться на WebRTC, если устройство не поддерживает SlimCore – например, при роуминге между разными платформами.

Что касается топологии, картина неоднозначна. Для групповых встреч Teams может использовать как SFU, так и MCU в зависимости от сценария и числа участников. В небольших встречах – вероятно, SFU. В крупных – возможно, каскад MCU или комбинация подходов. Документация Microsoft намеренно абстрагируется от этих деталей, предоставляя разработчикам API, а не описание архитектуры.

Важно отметить, что Teams не является чистым WebRTC-приложением. Нативная клиентская часть использует собственный медиа-стек, оптимизированный под Windows и macOS, что позволяет добиться более эффективного использования аппаратного ускорения и лучшего качества при слабом сигнале. WebRTC выступает как механизм резервирования (fallback) и как основа для браузерной версии.

Jitsi Meet [8] занимает особое положение. Это opensource-платформа, и её архитектура документирована настолько подробно, что она служит своего рода учебным пособием по построению WebRTC-конференций. Ключевой компонент – Jitsi Videobridge (JVB), представляющий собой WebRTC-совместимый SFU, то есть мультимедийный роутер.

JVB работает именно как SFU: он маршрутизирует видео- и аудиопотоки между участниками, не выполняя микширования и перекодирования. Это снижает нагрузку на сервер, позволяя одному экземпляру JVB обслуживать до сотен конференций одновременно. Для небольших групп из двух участников Jitsi может переключаться в P2P-режим, когда медиа идёт напрямую, экономя серверные ресурсы.

Серверная архитектура Jitsi Meet включает несколько компонентов: Prosody (XMPP-сервер для сигнализации), Coturn (TURN-сервер для NAT-traversal), и собственно JVB. JVB выставляет WebSocket-эндпоинт на порту 9090 и использует стандартный WebRTC-стек с ICE, DTLS и SRTP. Такая прозрачность делает Jitsi идеальным полигоном для изучения WebRTC-архитектур – и одновременно показывает, почему коммерческие платформы не раскрывают деталей: в opensource-мире видна каждая строка кода, и

оптимизации, составляющие конкурентное преимущество, скопировать тривиально.

В таблице Б.1 представлено сравнение разрабатываемой системы с аналогами по критериям.

Принципиальным архитектурным отличием разрабатываемой системы является гибридная маршрутизация медиаданных: участники малой группы обмениваются потоками напрямую без участия сервера, тогда как связь между группами осуществляется через сервер избирательной пересылки. Ни один из рассмотренных аналогов не реализует подобного механизма. Косвенным следствием гибридной архитектуры является частичная устойчивость к сбоям: установленные прямые соединения внутри группы сохраняются при недоступности серверного узла. Разрабатываемая система поддерживает полный набор режимов коммуникации – текстовый чат, голосовую и видеосвязь – и доступна в браузере, не уступая по этому критерию Jitsi Meet и Google Meet.

Вместе с тем разрабатываемая система уступает аналогам в части записи конференции и демонстрации экрана – функций, присутствующих в большинстве рассмотренных платформ. Реализация этих возможностей выходит за рамки поставленных задач и составляет направление дальнейшего совершенствования системы.

1.1.3. Метрики качества передачи данных в системах реального времени

Субъективное восприятие задаёт предельные значения, однако для мониторинга и управления необходим аппарат измеряемых метрик. Качество передачи в пакетных сетях оценивается тремя базовыми параметрами: задержкой, вариацией задержки (джиттером) и коэффициентом потерь пакетов.

Норматив ITU-T G.114 формулирует ограничение для целей сетевого планирования: односторонняя задержка не должна превышать 400 мс [9]. Если архитектура сети закладывает большую величину, пригодность для голосового трафика исключается на этапе проектирования.

Однако средняя задержка не отражает всей полноты картины. Вариация задержки – джиттер – становится ключевым параметром транспортного уровня в пакетных системах именно вследствие непредсказуемости времени прибытия отдельных пакетов [1]. Стек протоколов RTP выполняет функцию компенсации этой асинхронности, преобразуя поток пакетов с переменной задержкой в синхронный медиапоток. Для измерения джиттера RFC 3550 предлагает рекуррентную формулу скользящего среднего с коэффициентом 1/16, что обеспечивает сглаживание выбросов при низкой вычислительной стоимости [10].

Обратная связь о качестве реализуется через RTCP-отчёты, в которых получатель передаёт отправителю статистику: долю потерянных пакетов, кумулятивное количество потерь, межпакетный джиттер и временные метки [10]. На основе этих данных работают механизмы контроля перегрузки. Необходимо отметить, что сама обратная связь потребляет часть пропускной способности канала и вносит дополнительную задержку, то есть измеряющая подсистема неизбежно воздействует на измеряемую среду.

Потери пакетов не сводятся исключительно к ошибкам передачи. 1 обращает внимание на то, что информационные потери включают также деградацию, вносимую медиакодеками [1]. Сжатие видео с низким битрейтом приводит к потере информации до отправки пакетов в сеть, и стандартные метрики транспортного уровня эту составляющую не фиксируют.

Помимо транспортных метрик, качество восприятия определяется также параметрами кодирования медиапотока. Экспериментальные данные, полученные при тестировании видеосвязи через TURN-серверы, демонстрируют связь битрейта и пользовательского восприятия: для потока с разрешением 640×480 и частотой 24 кадра в секунду, закодированного в H.264, удовлетворительное качество достигается начиная примерно с 480 кбит/с; при битрейте выше 800 кбит/с дальнейшее улучшение становится малозаметным [11].

1.1.4. Требования к клиентским устройствам в системах реального времени

Текстовая и голосовая коммуникация предъявляют умеренные требования к клиентским устройствам, а вот передача видео, напротив, упирается в физические ограничения процессорных ресурсов, памяти и энергопотребления. Обязательный набор кодеков, зафиксированный в спецификациях, включает для видео VP8 (RFC 6386) и H.264 Constrained Baseline [2]; для аудио – Opus (RFC 6716), PCMA/PCMU (G.711) и генератор комфортного шума по RFC 3389 [12]. Кодирование и декодирование этих форматов в реальном времени с минимальной задержкой создаёт значительную вычислительную нагрузку.

В полносвязной P2P-сети (mesh) каждый участник отправляет свой поток всем остальным и принимает потоки от всех остальных. В конференции из трёх человек каждое устройство поддерживает два пиринговых соединения, но с ростом числа участников проблема не ограничивается дублированием потоков в сети. Исследователи, разрабатывавшие систему SnoW, подчёркивают, что основным узким местом становятся отдельные стеки рендеринга, потребляющие память и процессорное время в браузере [13]. Именно клиентское устройство, а не пропускная способность сети, часто оказывается лимитирующим фактором масштабирования.

При использовании клиент-серверной архитектуры сервер берёт на себя декодирование всех входящих потоков, композицию общего видеокадра и кодирование результирующего потока [14]. Нагрузка на клиента минимизируется, однако сервер должен обладать значительной вычислительной мощностью.

Неравенство участников, в плане мощности их устройств, является фундаментальным свойством реальных конференций: пиры различаются пропускной способностью, процессорными ресурсами и уровнем заряда батареи и так далее [15]. Динамическое управление P2P-топологией пытается учитывать эту неравномерность. Один из предложенных механизмов – promotion – перемещает пира с высокой пропускной способностью и активной речью ближе

к корню дерева распределения, сокращая высоту дерева и задержку между активными участниками [15]. Однако такое перестроение требует непрерывного мониторинга и реконфигурации, создавая дополнительную нагрузку на те же устройства.

Таким образом, видео в реальном времени предъявляет комплекс требований, объединяющих интенсивное кодирование и декодирование, синхронизацию с аудио, адаптацию к изменяющейся полосе, дублирование потоков для нескольких получателей и энергетические ограничения мобильных устройств.

1.2. Протоколы передачи данных в реальном времени

1.2.1. WebSocket и Socket.IO

Определение из RFC 6455 характеризует WebSocket как протокол, обеспечивающий двустороннюю связь между клиентом, исполняющим непроверенный код в контролируемом окружении, и удалённым хостом, явно разрешившим такую коммуникацию [16]. WebSocket не является конкурентом HTTP, несмотря на распространённое противопоставление. Протокол намеренно спроектирован так, что стартовое рукопожатие совместимо с HTTP, и оба протокола могут сосуществовать на одном порту, взаимодействуя с промежуточными прокси без их модификации [16]. Таким образом, смена модели взаимодействия сопровождается сменой модели доверия.

Ключевое техническое преимущество перед HTTP-моделью – полнодуплексный обмен данными без периодических запросов, что снижает накладные расходы и задержку [17]. Для текстового чата разница в задержке между long-polling (длительным опросом) и WebSocket может быть несущественной, но для сигнализации в WebRTC, где каждая миллисекунда оттягивает установку медиасоединения, она становится критичной.

Socket.IO представляет собой библиотеку, надстраивающуюся над WebSocket и предоставляющую резервный механизм на HTTP long-polling.

Экспериментальные данные показывают, что при нагрузке в 1000 клиентов Socket.IO демонстрирует среднюю пропускную способность около 27152 сообщений в секунду при времени пути туда и обратно порядка 31 мс – показатели ниже, чем у чистого WebSocket [17]. Это различие объясняется дополнительным протокольным уровнем: Socket.IO добавляет собственные пакеты поддержания соединения и служебные сообщения. Однако резервный механизм остаётся востребованной в корпоративных сетях, где прокси-серверы могут блокировать WebSocket-трафик.

Применительно к сигнализации в WebRTC WebSocket выступает транспортом для обмена SDP-предложениями и ICE-кандидатами. Исследователи отмечают, что как Socket.IO, так и WebSocket могут использоваться для сигнализации, при этом WebSocket является базовым транспортом для обоих подходов [14].

1.2.2. Message Queue Telemetry Transport

Если WebSocket, предоставляет транспортный канал, то MQTT (Message Queue Telemetry Transport) предлагает принципиально иную модель взаимодействия – асинхронную публикацию-подписку. Протокол изначально создавался как «лёгкий протокол очередей сообщений и транспорта, подходящий для передачи телеметрических данных в сценариях M2M, WSN и IoT» [18]. Его ключевые характеристики прямо вытекают из этой ниши: «асинхронная коммуникационная модель с событиями, низкие накладные расходы (заголовок от 2 байт), модель публикации/подписки, отделяющая производителя данных от потребителя через тематические очереди, и общая ориентация на реализации с низким энергопотреблением и малым объёмом кода» [18].

В архитектуре MQTT присутствуют две роли: клиенты, выступающие издателями или подписчиками, и брокер, выполняющий функции посредника. Издатели отправляют сообщения брокеру, а тот определяет получателей на основе зарегистрированных подписок на определённые темы (topics) [19]. Таким

образом достигается полная временная и пространственная развязка: издателю не нужно знать ни адресов подписчиков, ни факта их присутствия в момент публикации. Это фундаментальное отличие от клиент-серверной модели WebSocket, где «сессия инициируется без обращения к публикации/подписке, вместо этого она опирается на рукопожатие от клиента к серверу для начала общения» [19].

Для повышения надёжности на ненадёжных каналах MQTT добавляет три уровня качества обслуживания (QoS) поверх TCP. Уровень 0 – не более одной доставки (без подтверждения), уровень 1 – по крайней мере одна доставка (с подтверждением), уровень 2 – ровно одна доставка (с четырёхсторонним рукопожатием) [18]. Эти механизмы покрывают сценарии, в которых разрыв TCP-соединения может приводить к потере сообщений в процессе передачи, что критично для телеметрии.

Экспериментальные сравнения MQTT и WebSocket в контексте передачи потоковых данных IoT показывают неоднозначную картину. С одной стороны, «MQTT может передавать больше данных в единицу времени (более высокая пропускная способность), потребляет меньше ресурсов CPU и лишь незначительно уступает WebSocket по задержке» [20]. С другой – сама модель Pub/Sub оказывается архитектурно предпочтительнее для распределённых сенсорных сетей, где устройства могут находиться в труднодоступных местах, а ожидание доступности клиента или сервера, как в WebSocket, было бы губительным [20]. Малый фиксированный заголовок (минимум 2 байта) и поддержка многоадресной рассылки (один-ко-многим) делают MQTT естественным выбором для систем с жёсткими ограничениями по энергопотреблению и пропускной способности [19].

Однако те же самые достоинства, которые делают MQTT идеальным транспортом для телеметрии, ограничивают его применимость в видеоконференциях. Протокол оптимизирован для коротких сообщений-событий и не предназначен для непрерывной потоковой передачи медиаданных с жёсткими требованиями по джиттеру и синхронизации. MQTT может служить

вспомогательным каналом сигнализации – например, для рассылки уведомлений о состоянии участников или управления сессией в распределённой системе – но не способен заменить RTP-потоки. Поэтому в архитектурах реального времени MQTT занимает нишу событийной шины, а не магистрального транспорта.

1.2.3. SIP

SIP – протокол прикладного уровня для установления, изменения и завершения мультимедийных сессий; он не передаёт голос и видео, а только управляет сессией, оставляя транспорт медиа отдельным каналам (как правило, RTP) [21]. Таким образом, если MQTT и WebSocket – это транспортные механизмы, то SIP – управляющий контур поверх них.

Архитектура SIP строится вокруг пользовательского агента (User Agent, UA), который в одном запросе выступает инициатором (UAC), а в другом – обработчиком (UAS); между ними располагаются SIP-прокси (пересылают запросы дальше, stateful-прокси дополнительно разветвляют запрос на несколько адресатов, реализуя механизм forking), Registrar-сервер (регистрирует текущий URI абонента через REGISTER, позволяя маршрутизировать вызовы независимо от IP) и Redirect-сервер (возвращает адресатов в 3xx-ответе, перекладывая повторный запрос на UAC) [21]. Такая инфраструктура даёт отказоустойчивые схемы с балансировкой и географически распределёнными узлами.

Установление сессии строится как трёхэтапное рукопожатие INVITE – 200 OK – ACK, формирующее SIP-диалог – состояние, разделяемое двумя UA на протяжении всей сессии. Именно диалог (а не stateless-обмен) отличает SIP от MQTT-подобных протоколов: внутри диалога агенты помнят параметры сессии и коррелируют последующие запросы; re-INVITE позволяет изменить параметры медиа «на лету» (добавить видеопоток, переключить кодек, поставить вызов на удержание) без разрыва соединения [21]. Поскольку SIP может работать поверх UDP, протокол реализует надёжность через повторные передачи с удвоением интервалов; это даёт устойчивость к потерям, но добавляет задержку установки сессии по сравнению с TCP-транспортом.

Параметры медиасессии в SIP не описываются самостоятельно: в тело сообщения встраивается Session Description Protocol (SDP), а их согласование выполняется по модели Offer/Answer [22]: одна сторона предлагает перечень поддерживаемых медиавозможностей, другая – отвечает согласованным подмножеством. SDP описывает тип медиа, транспорт, кодеки, адрес медиаконечной точки и направление потока [23]. Именно этот механизм впоследствии был заимствован WebRTC для согласования медиа через JSEP – протокольное наследие SIP проросло в современный веб; отличие лишь в том, что SDP-тело в WebRTC транспортируется не через SIP-сообщения, а через произвольный сигнальный канал, что даёт разработчику свободу выбора, недоступную в классической SIP-инфраструктуре.

WebRTC не предписывает обязательный сигнальный протокол [3], однако совместимость с SIP-инфраструктурой (корпоративные IP-АТС, PSTN-шлюзы) остаётся практически значимой. Для этой задачи разработан WebSocket-транспорт для SIP [24], позволяющий браузеру стать полноценным SIP User Agent: SIP-сообщения инкапсулируются в WebSocket-фреймы и доставляются через стандартный HTTP-порт 80/443, что устраняет блокировку SIP-трафика на корпоративных прокси. Подтверждено, что такая схема обеспечивает полноценный UA без плагинов, сохраняя совместимость с существующими SIP-регистраторами и прокси-серверами [25]. Архитектурно это воспроизводит то же разделение ответственности, что и в 1.2.1: WebSocket – транспорт, SIP – управляющая логика поверх него.

Обратная сторона – двойная сигнальная нагрузка: параметры WebRTC (ICE-кандидаты, DTLS-отпечатки) транспортируются внутри SDP-тел SIP-сообщений, а сам SIP путешествует по WebSocket. Там, где совместимость с SIP-инфраструктурой не требуется, WebSocket-based сигнализация предпочтительнее: она проще, не несёт накладных расходов SIP-транзакционной машины и легко расширяется произвольными событиями. Именно поэтому в рамках данной работы сигнализация реализована через Socket.IO, а не через SIP/WebSocket: для замкнутой системы с нестандартной ячеистой

маршрутизацией и алгоритмом выбора передатчика сигнальный протокол должен отражать собственную логику системы, а не подчиняться ограничениям телефонного стека, проектировавшегося для принципиально иных сценариев.

1.2.4. RTP, SRTP и RTCP

Если сигнальные протоколы решают задачу координации участников, то транспортировка собственно медиаданных в системах реального времени опирается на Real-time Transport Protocol (RTP), определённый в RFC 3550. RTP работает поверх UDP (хотя способен функционировать и поверх TCP) и обеспечивает доставку аудио- и видеопотоков с временными метками и порядковыми номерами пакетов, что позволяет приёмнику восстанавливать синхронизацию и обнаруживать потери [26].

Эта базовая функциональность критична для сценариев, где данные передаются в режиме реального времени: порядковый номер даёт возможность выстроить пакеты в правильной последовательности независимо от порядка их прибытия, а временная метка – воспроизвести их с нужной скоростью и синхронизировать аудио с видео. Именно RTP является тем слоем, на котором работают все медиакодеки, рассмотренные в разделах 1.2.3 и 1.2.4.

Параллельно с RTP действует протокол управления RTCP (Real-time Control Protocol). Он предоставляет обратную связь о качестве передачи: отчёты отправителя и получателя содержат долю потерянных пакетов, накопленное число потерь, межпакетный джиттер и временные метки, позволяя алгоритмам контроля перегрузки адаптировать битрейт [26]. Эта обратная связь создаёт замкнутый контур управления, в котором измеряющая подсистема неизбежно потребляет часть полосы пропускания, но без неё динамическая адаптация к изменяющимся сетевым условиям была бы невозможна.

Безопасность медиатрафика в WebRTC обеспечивается Secure Real-time Transport Protocol (SRTP). Исследователи, анализировавшие трафик Google Hangouts, подтверждают, что «безопасность протокола WebRTC основывается на двух составляющих: с одной стороны, используется протокол Datagram

Transport Layer Security (DTLS), с другой – Secure Real-time Transport Protocol (SRTP) применяет алгоритм AES-256 для шифрования всех медиакоммуникаций» [26]. Ключи для SRTP извлекаются в процессе DTLS-рукопожатия, что позволяет устанавливать защищённый медиаканал без предварительного обмена секретами через сигнальный сервер. Это архитектурное решение избавляет сигнальный канал от необходимости передавать ключи, снижая требования к доверию к промежуточным узлам сигнализации.

Важно отметить, что SRTP защищает только содержимое пакетов, оставляя заголовки RTP открытыми для маршрутизации. В архитектурах с SFU, где сервер перенаправляет пакеты без декодирования, это свойство оказывается принципиальным: SFU может селективно пересылать RTP-пакеты, анализируя их SSRC и временные метки, не имея доступа к зашифрованным медиаданным.

1.2.5. WebRTC

WebRTC представляет собой открытый фреймворк, поддерживаемый многими браузерами, который обеспечивает низколатентный обмен медиа без необходимости в центральном медиасервере [27]. Медиатрафик передаётся напрямую между пирами, однако для сигнализации сервер всё же требуется. На рисунке 1.1 показан принцип работы WebRTC.

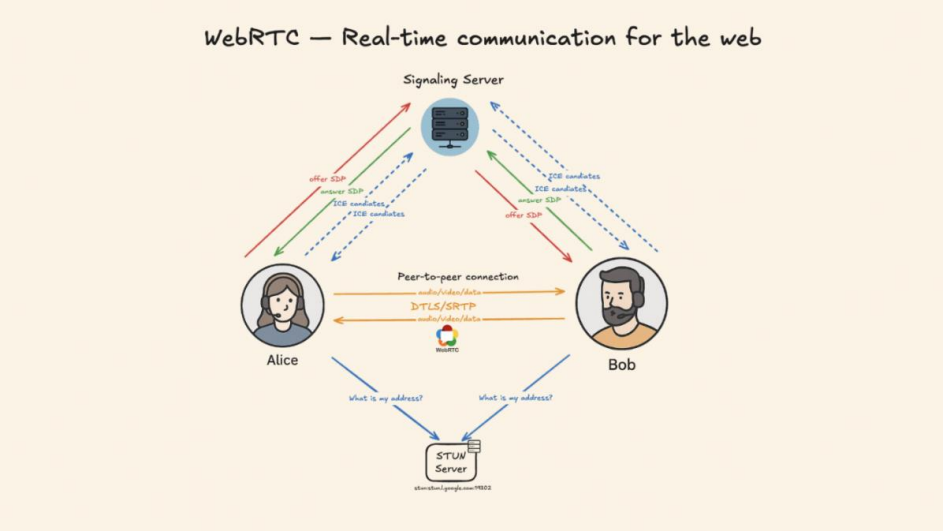


Рисунок 1.1 – Принцип работы WebRTC

Формальное определение ICE, зафиксированное в RFC 8445, описывает его как протокол для обхода NAT (Network Address Translator) при коммуникации на основе UDP, использующий STUN и его расширение TURN. Приведённая формулировка хотя и точна, однако описывает сложный механизм согласования соединения, который в реальном времени перебирает возможные сетевые маршруты и выбирает из них оптимальный.

Процесс начинается со сбора кандидатов – потенциальных IP-адресов и портов, по которым участник может быть доступен. RFC 8445 выделяет три типа кандидатов: host (адрес на локальном сетевом интерфейсе), server reflexive (адрес, полученный от STUN-сервера – то есть тот публичный адрес и порт, которые NAT выделил для исходящего запроса / кандидат от сервера) и relay (адрес, арендованный у TURN-сервера, через который медиатрафик будет ретранслироваться / ретранслятор). Логика здесь проста: чем меньше промежуточных узлов, тем ниже задержка, поэтому приоритет всегда отдаётся прямым соединениям – host предпочтительнее кандидата от сервера, а кандидат предпочтительнее ретранслятора. Но простота эта обманчива: реальная сеть может сделать недоступным любой из этих вариантов.

Собрав кандидатов, стороны обмениваются ими через сигнальный канал. Именно здесь WebSocket или Socket.IO, описанные в разделе 1.2.1, играют критическую роль: пока ICE-кандидаты не доставлены, пиры даже не знают, по каким адресам им пытаться связаться. Исследователи, описывающие интеграцию WebRTC и WebSocket, подчёркивают: сигнальный WebSocket используется не только для SDP, но и для передачи ICE-кандидатов, а затем – для управляющих сообщений в течение всей сессии. Медиа и сигнализация идут по разным каналам, и это архитектурное решение избавляет браузер от необходимости разбирать SDP или управлять ICE-таймерами.

Когда обе стороны получили полный набор кандидатов, начинается фаза проверки связности (connectivity checks). Пиры попарно тестируют комбинации своих кандидатов, отправляя STUN-запросы и ожидая ответов. RFC 8445 описывает этот процесс как последовательность проб, в которой проверяемые

пары сортируются по приоритету, а успешно ответившие – номинируются как рабочие. Слово «номинация» здесь неслучайно: ICE не просто фиксирует первый сработавший маршрут, а продолжает проверять альтернативы в поисках наилучшего. Если через некоторое время обнаруживается более приоритетная пара, она заменяет текущую. Это обеспечивает адаптивность механизма ICE, который способен переключиться с соединения-ретранслятора на прямое, если NAT вдруг позволит [28].

STUN: обнаружение публичного адреса.

STUN – самый лёгкий и одновременно самый непонятый из трёх компонентов. Его часто воспринимают как «протокол для обхода NAT», однако RFC 5389 с самого начала предупреждает: STUN не является самостоятельным решением для преодоления NAT – это инструмент, который должен использоваться в составе более широкой стратегии. Ключевое слово здесь – «инструмент».

Принцип работы STUN достаточно прост. Клиент отправляет Binding Request на публичный STUN-сервер, тот фиксирует IP-адрес и порт, с которых пришёл запрос (а они уже преобразованы NAT'ом), и возвращает их в ответе. Клиент узнаёт свой публичный адрес. Этот адрес становится ICE-кандидатом типа server reflexive. RFC 5389 также отмечает, что Binding-метод может использоваться для поддержания привязки (keep-alive), не давая NAT'у сбросить трансляцию по тайм-ауту [28].

Ограничения STUN становятся очевидны при рассмотрении различных типов NAT. Наиболее распространённые варианты – full cone, restricted cone и port-restricted cone – позволяют STUN-адресу, полученному одним хостом, быть использованным другим хостом для установления входящего соединения (иногда с оговорками). Но симметричный NAT (а это типичная ситуация для корпоративных сетей и мобильного интернета) назначает разное внешнее отображение для каждого адреса назначения. STUN-запрос к STUN-серверу открывает одну трансляцию; попытка другого хоста отправить пакет на этот

адрес натыкается на другую трансляцию и терпит неудачу. В такой ситуации без TURN не обойтись.

TURN: ретранслятор.

Если STUN предназначен для определения публичных параметров соединения, то TURN используется в случаях, когда прямое соединение установить невозможно. RFC 5766 определяет TURN как протокол, позволяющий хосту за NAT (TURN-клиенту) запросить другой хост (TURN-сервер) выступить в роли ретранслятора пакетов. Все медиаданные идут через TURN-сервер, который пересылает их от одного клиента другому. Это добавляет и задержку, и нагрузку на серверную инфраструктуру – но связь появляется там, где иначе её не было бы.

Важная деталь, отличающая TURN от тривиального прокси: один релейный адрес может обслуживать несколько удалённых пиров. RFC 5766 подчёркивает, что TURN позволяет клиенту общаться с множеством пиров через один адрес-ретранслятор, что существенно экономит ресурсы сервера по сравнению с наивным подходом. Кроме того, TURN изначально проектировался для использования в составе ICE, хотя и может применяться независимо.

Исследователи, тестировавшие децентрализованную TURN-маршрутизацию в рамках проекта WebRTC Swarms, приводят впечатляющие цифры: эмпирическая оценка показала почти 100-процентный успех NAT-обхода при использовании инцентивизированной децентрализованной TURN-ретрансляции – против примерно 85% для подходов, полагающихся только на STUN. Разрыв в 15 процентных пунктов – это именно та доля пользователей, которые без TURN просто не смогли бы соединиться. Для коммерческих систем видеоконференций это существенно влияет на доступность сервиса для пользователей.

В случае симметричных NAT, где адрес, полученный через STUN, не работает для других хостов, задействуется TURN – протокол, позволяющий узлу за NAT запросить ретрансляцию трафика через TURN-сервер [29]. Этот путь добавляет задержку и нагрузку на сервер, но обеспечивает связность, когда

прямые соединения невозможны. Процесс описывается как последовательность: сбор кандидатов, их распределение через сигнальный сервер и попытка соединения с NAT-traversal [30]. На рисунке 1.2 показан принцип работы STUN и TURN.

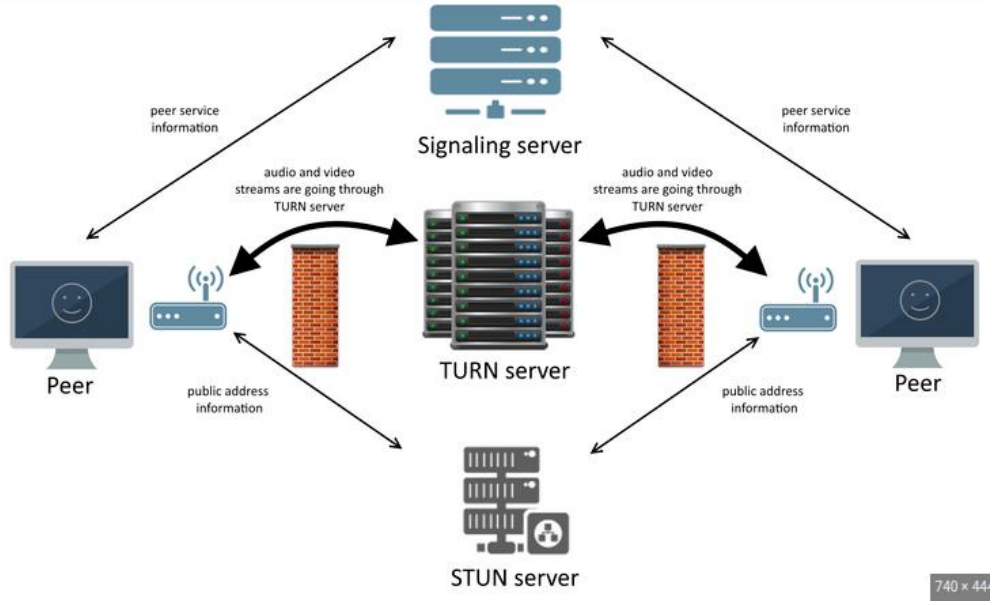


Рисунок 1.2 – Принцип работы STUN и TURN механизмов

Связка ICE-STUN-TURN как единый механизм.

Возвращаясь к общему процессу, можно подвести промежуточный итог. ICE – это не сумма трёх протоколов, а именно фреймворк, координирующий их взаимодействие. STUN поставляет публичные адреса (server reflexive candidates), TURN – релейные адреса (relay candidates), а сам ICE организует их сбор, обмен, проверку и выбор. В распределённой архитектуре, описанной в [27], эта же связка работает на уровне супер-узлов: ICE обеспечивает установление соединений между супер-узлами через NAT и файрволы, используя STUN и TURN для обхода, причём TURN служит резервным механизмом, когда прямые соединения недоступны [27].

Практическая рекомендация, зафиксированная в сообществе: STUN (а возможно, и TURN) необходим каждому WebRTC-участнику за NAT. Без STUN браузер не сможет определить свой публичный адрес. Без TURN часть пользователей останется без связи. Именно поэтому практически все production-

развёртывания WebRTC включают оба компонента: один-два публичных STUN-сервера (google, cloudflare) и собственный TURN-сервер (чаще всего coturn) с настроенной аутентификацией.

SDP (Session Description Protocol) используется для обмена описаниями сессий, содержащими информацию о поддерживаемых кодеках, разрешениях и битрейтах [30]. В архитектуре JSEP, описанной в RFC 8829, при необходимости обмена offer/answer инициирующая сторона создаёт предложение через createOffer, после чего приложение устанавливает локальную конфигурацию через setLocalDescription, не интерпретируя SDP [31]. Разделение ответственности – браузер управляет медиа, приложение управляет сигнализацией – является базовым принципом WebRTC. На рисунке 1.3 показан принцип работы ICE, STUN, TURN в связке.

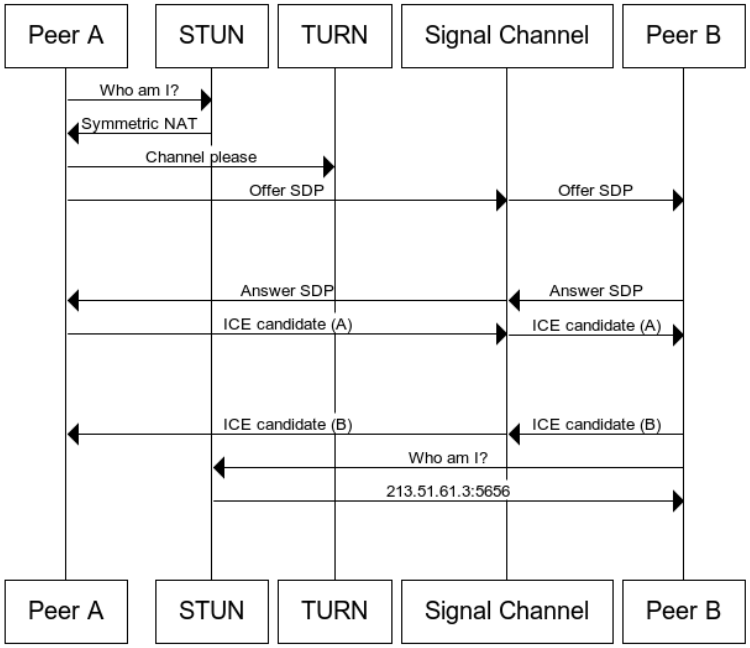


Рисунок 1.3 – Работа ICE, STUN, TURN в связке

Передача голоса в WebRTC.

Спецификация RFC 7874 устанавливает обязательный набор аудиокодеков для WebRTC: Opus с форматом полезной нагрузки по RFC 7587, а также PCMA и PCMU, описанные в G.711 [12]. Opus представляет собой адаптивный кодек с диапазоном битрейтов от 6 до 510 кбит/с, способный подстраиваться под сетевые

условия. PCMA/PCMU обеспечивают совместимость с телефонными шлюзами и legacy-системами, где импульсно-кодовая модуляция остаётся основным форматом.

Отдельного рассмотрения заслуживает Comfort Noise (CN). В традиционной телефонии фоновый шум в паузах является индикатором работоспособности канала. WebRTC позволяет генерировать CN на стороне получателя по спецификации RFC 3389, однако для Opus рекомендуется использовать собственный механизм CN, а применение RFC 3389 с Opus не рекомендовано [12].

Эхоподавление (AEC) входит в перечень рекомендованных компонентов обработки: конечные точки должны включать акустический эхокомпенсатор или иную форму контроля эха [12]. При задержке даже в 50 мс эхо становится заметным, а при 150 мс – существенно мешает разговору. Автоматическая регулировка усиления (AGC) нормализует уровень сигнала при вариациях громкости речи и расстояния до микрофона; рекомендованный фильтр для нормализации – фильтр Баттерворта второго порядка с частотой среза 300 Гц [12].

Обнаружение голосовой активности (VAD) и прерывистая передача (DTX) оставлены опциональными [12]. Причина – эвристическая природа VAD и риск обрезания начала фразы при сложных условиях джиттера. В интернете с переменной задержкой задача надёжного детектирования голоса усложняется по сравнению с телефонными сетями.

Передача видео в WebRTC.

Видеокодеки в WebRTC представлены обязательными VP8 (RFC 6386) и H.264 Constrained Baseline [2]. VP8 – открытый кодек, H.264 – индустриальный стандарт с аппаратной поддержкой в большинстве чипсетов. Выбор между ними на практике определяется доступностью аппаратного ускорения, что особенно важно для мобильных устройств с ограниченным энергопотреблением.

Управление перегрузкой реализуется через механизм Google Congestion Control (GCC), который пытается обнаружить перегрузку до возникновения

потерь, анализируя межинтервальные задержки последовательных пакетов [32]. При нарастании задержки алгоритм интерпретирует это как образование очереди в сети и снижает битрейт; при стабильности задержки – пробует повысить. Такой подход, основанный на задержке, а не на потерях, позволяет реагировать на изменения канала с упреждением.

Для многопользовательских сценариев применяется simulcast: каждый участник отправляет несколько версий потока с разным качеством (низким, средним, высоким), а сервер выбирает, какую версию переслать конкретному получателю в зависимости от его сетевых условий [32]. Таким образом, адаптация к условиям приёма осуществляется без перекодирования на сервере.

Согласование размеров видео на этапе сигнализации происходит через SDP-атрибут, с помощью которого получатель может указать поддерживаемые разрешения [31]. Это позволяет отправителю не генерировать поток, заведомо превышающий возможности приёмника.

1.2.6. Современные протоколы медиадоставки

Классические протоколы адаптивного стриминга, такие как HLS и DASH, изначально проектировались с прицелом на устойчивость и масштабирование ценой высокой задержки, измеряемой десятками секунд. Видеоконференции с такими задержками неприемлемы, что и обусловило доминирование WebRTC в этой нише. Однако в последние годы появились эволюционные расширения этих протоколов, направленные на сокращение задержки до величин, сопоставимых с интерактивным общением.

Низколатентные версии HLS и DASH (LL-HLS и LL-DASH) добиваются этого путём дробления традиционных сегментов на чанки. «Низколатентный HLS и низколатентный DASH используют новый процесс кодирования и передачи, эффективно разбивая каждый сегмент на несколько (как правило, 4–10) чанков и передавая их по мере готовности. Поскольку каждый чанк значительно короче сегмента, задержка в системе потоковой передачи сокращается» [33]. Экспериментальные замеры показывают, что «с LL-DASH

плеерами возможна потоковая передача с задержками порядка 3–4 секунд, тогда как с LL-HLS потоками достижимы задержки 6–8 секунд» [33]. Это на порядок лучше классических реализаций, но всё ещё далеко от 150–400 мс, требуемых для естественного разговора согласно ITU-T G.1010 [1]. Таким образом, LL-HLS и LL-DASH находят применение в сценариях с поточной доставкой «почти реального времени» – вебинары, спортивные трансляции с минимальным отставанием, – но не могут заменить WebRTC в диалоговых видеоконференциях.

Более радикальное снижение задержки обещает семейство протоколов, объединённых вокруг HTTP/3 и QUIC. QUIC – новый транспортный протокол, реализованный на прикладном уровне поверх UDP, – устраняет проблему блокировки головы очереди (head-of-line blocking), свойственную TCP, и тем самым обеспечивает «более плавную передачу данных и снижает сквозную задержку» [34]. Поверх QUIC построен HTTP/3, а спецификация WebTransport, в свою очередь, «включающая HTTP/3 и QUIC в качестве фундаментальных компонентов, открывает новые возможности для улучшения сервисов доставки медиа, таких как облачный гейминг, видеоконференции и рендеринг в реальном времени» [35]. WebTransport обеспечивает мультиплексированную, двунаправленную, низколатентную передачу данных между серверами и клиентами, позволяя одновременно управлять надёжными и ненадёжными потоками – именно то, что необходимо для раздельной доставки аудио, видео и управляющих сообщений [35].

Цифры подтверждают потенциал подхода: «наша система на базе WebTransport способна достигать разумной сквозной задержки порядка 115 мс» [35]. Это значение уже попадает в целевой диапазон для голосовых разговоров, определённый ITU-T G.1010 [1], и сопоставимо с показателями хорошо настроенной WebRTC-сессии.

Параллельно IETF развивает протокол Media over QUIC (MoQ), цель которого – «предложить технологию потоковой передачи, обеспечивающую одновременно масштабируемость и интерактивность» [34]. MoQ опирается на QUIC и наследует все его преимущества, включая устранение блокировок

очередей, быстрое переключение сетевых путей и встроенное шифрование. «Одним из преимуществ этого нового транспортного протокола является его потенциал для низкой задержки, обусловленный использованием UDP и устранением задержек в очередях» [35]. В отличие от WebRTC, который несёт багаж совместимости с классической телефонией, MoQ проектируется как чистый медиатранспорт для эпохи HTTP/3, потенциально способный обслуживать как потоковое видео по запросу, так и интерактивные конференции в рамках единой инфраструктуры.

Следует, однако, подчеркнуть, что WebTransport и MoQ находятся на стадии активной стандартизации и внедрения, тогда как WebRTC уже развёрнут в миллиардах браузеров и прошёл многолетнюю проверку практикой. Их появление не отменяет WebRTC, а скорее очерчивает контуры следующего поколения протоколов реального времени, в котором транспортный уровень на базе QUIC постепенно заменит связку ICE–STUN–TURN, а сигнализация, возможно, переедет на MoQ-специфичные механизмы. Для задач данной работы существенно то, что любая распределённая архитектура, проектируемая сегодня, должна учитывать этот вектор эволюции и по возможности избегать жёсткой привязки к конкретному транспортному стеку, предусматривая шлюзы или абстракции для будущей интеграции.

1.3. Архитектура многопользовательских коммуникационных систем

1.3.1. Полносвязная P2P-сеть

Полносвязная P2P-сеть (mesh) представляет собой наиболее прямой способ организации групповой коммуникации: каждый участник устанавливает соединение с каждым другим и отправляет медиапотоки напрямую [36]. Инициатор начинает с двухстороннего вызова, а затем последовательно приглашает остальных.

Ключевое ограничение mesh заключается в квадратичном росте числа соединений. Каждый узел поддерживает $N - 1$ соединение, а общее количество

соединений составляет $N(N-1)/2$. Разработчики системы SnoW отмечают, что модель плохо масштабируется за пределами 4–6 участников, причём узким местом является не только дублирование потоков в сети, но и ресурсы клиентского устройства: отдельные стеки рендеринга потребляют память и процессорное время [13]. Хотя полносвязная топология обеспечивает минимальную задержку на пути медиа, сетевая нагрузка на каждом узле растёт линейно по числу потоков: каждый узел принимает $N-1$ и отправляет $N-1$ медиапотоков [14]. Для малых групп mesh остаётся эффективным решением благодаря нулевым серверным затратам и отсутствию единой точки отказа, однако масштабирование ограничено физическими характеристиками клиентских устройств. На рисунке 1.4 показана полносвязная P2P.

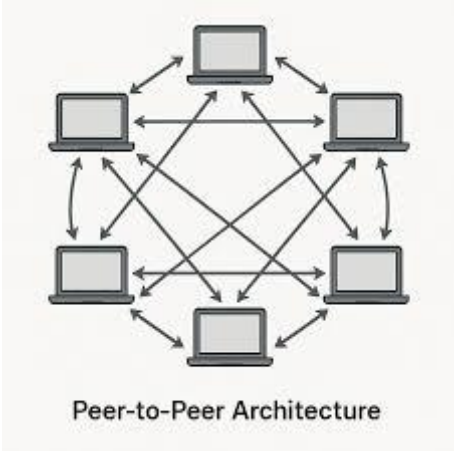


Рисунок 1.4 – Полносвязная P2P

1.3.2. Клиент-серверные архитектуры

При увеличении числа участников на смену mesh приходят серверные архитектуры. MCU (Multipoint Control Unit) принимает потоки от всех участников, смешивает их и отправляет каждому единый композитный поток [37]. Клиент получает один поток вне зависимости от размера группы, что минимизирует нагрузку на него, однако сервер вынужден выполнять декодирование, композицию и повторное кодирование. MCU требует значительных вычислительных ресурсов, а обработка вносит дополнительную задержку [36].

Альтернативой является SFU (Selective Forwarding Unit) – сервер, который не микширует потоки, а селективно перенаправляет их участникам [36]. В терминологии RFC 7667 такой элемент называется Selective Forwarding Middlebox (SFM) – промежуточное устройство, способное выбирать, какие из активных источников будут отправлены каждому конечному узлу, сохраняя при этом оригинальные идентификаторы SSRC [37]. На рисунке 1.5 показаны виды серверных архитектур.

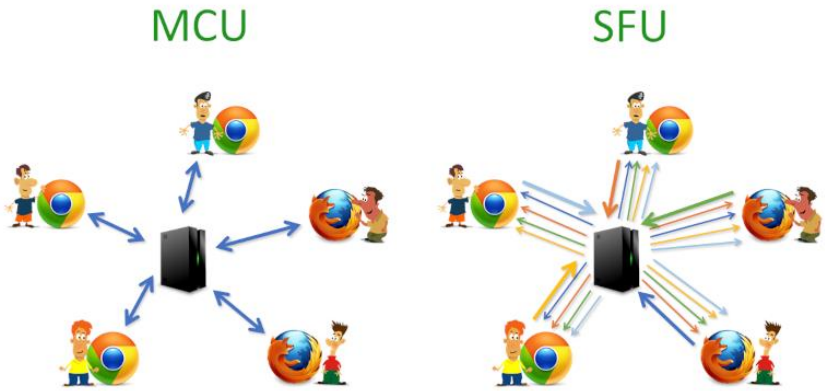


Рисунок 1.5 – Серверные архитектуры

SFU не выполняет декодирования и кодирования, что делает его значительно легче MCU [32]. Экспериментальные данные, полученные при разработке модуля видеоконференций для LMS, демонстрируют разницу: для конференции из четырёх участников загрузка процессора сервера MCU достигает 100%, тогда как SFU практически не нагружает процессор, однако клиентская нагрузка при SFU выше [46]. SFU хорошо масштабируется и подходит для средних и больших групп, а MCU сохраняет применимость для случаев со слабыми клиентскими устройствами [36].

1.3.3. Гибридные топологии

Гибридные архитектуры возникают из стремления объединить сильные стороны различных подходов. Одно из направлений – децентрализованная сигнализация: проект WebRTC Swarms предлагает архитектуру, интегрирующую «луковую»-маршрутизацию, доказательства с нулевым

разглашением и криптоэкономические стимулы, исключая центральный сигнальный сервер [47]. Медиатрафик при этом остаётся пиринговым.

Другое направление – иерархическая медиа-маршрутизация. Исследователи предлагают метод построения сети, состоящей из распределительных деревьев на уровне отдельных пиров и опорной сети, связывающей эти деревья [15]. Такой подход воспроизводит на прикладном уровне структуру, аналогичную сегментированным сетям передачи данных.

В области стриминга используется модель гибридного CDN-P2P: CDN-серверы обслуживают первых запрашивающих пиров, после чего накопившие ёмкость пиры берут на себя обслуживание остальных, освобождая сервер [48]. Метод двухэтапной кластеризации (Two-Way/Hybrid clustering) формирует кластеры по интересам, а затем адаптивно перестраивает их с учётом географического расположения и поведения пиров [48].

Модель, комбинирующая WebRTC для медиа и WebSocket для сигнализации, также может рассматриваться как гибридная: пиринговое медиа соединяется с клиент-серверной сигнализацией и резервным каналом [49]. Таким образом, общим для гибридных подходов является отказ от архитектурного догматизма и выбор топологии в зависимости от ограничений конкретного сценария.

1.3.4. Сравнительный анализ архитектур

Сравнение архитектур требует формализации критериев. По параметру upload-нагрузки на участника: в mesh каждый пир отправляет $N - 1$ потоков с суммарным битрейтом, пропорциональным размеру группы; в SFU и MCU – ровно один поток [13]. На асимметричных каналах это делает mesh неприменимым уже при 4–5 участниках.

Серверная стоимость распределяется противоположным образом: P2P не требует медиасервера; SFU требует сервер с умеренной вычислительной нагрузкой, пропорциональной числу пересылаемых потоков; MCU требует мощного сервера для декодирования и микширования. Сравнительная таблица в

обзорной работе [36] характеризует mesh как архитектуру с плохой масштабируемостью и высокой клиентской нагрузкой, MCU – с высокой серверной нагрузкой и хорошей масштабируемостью, SFU – как сбалансированный вариант с хорошей масштабируемостью.

По задержке P2P имеет преимущество за счёт прямых путей; SFU добавляет один хоп; MCU добавляет хоп и задержку обработки. Однако в глобальных сетях с участием TURN разница между mesh и SFU часто нивелируется.

Масштабируемость характеризуется несколькими метриками. Эксперименты с адаптивной multi-hop (многоскачковой) P2P-системой показали, что среднее время присоединения к группе из 10 участников составляет 104 мс, а к группе из 100 – 1101 мс [27]. Время отклика при смене говорящего для группы из 10 участников – 150 мс, для группы из 100 – 550 мс [27]. На таблице Б.2 можно увидеть сравнение архитектур.

Таким образом, P2P пригоден для малых групп с однородными участниками; SFU – для большинства практических сценариев с десятками участников; MCU – для слабых клиентов, не способных обрабатывать множественные потоки. При этом ни одна из архитектур не решает одновременно проблем масштабирования, устойчивости и гетерогенности. Фиксированный центральный SFU остаётся единой точкой отказа; mesh неустойчив при росте группы; MCU требует дорогостоящей серверной инфраструктуры; гибридные модели, как правило, привязаны к конкретным сценариям и не предлагают универсального механизма сегментации. Эти ограничения создают предпосылки для перехода к распределённым ячеистым архитектурам, в которых функции маршрутизации и управления распределяются между супер-пирами, а кластеры (ячейки) соединяются через межсегментные каналы без перекодирования.

1.4. Ячеистые топологии и распределённое управление

1.4.1. Сотовый принцип в телекоммуникациях

Идея ячеистой структуры заимствована из сотовой телефонии: пространство разбивается на зоны, внутри которых связь управляется локально, а между зонами осуществляется через магистральные соединения. В контексте данной работы под ячейкой понимается группа пиров, объединённых локальной связностью и общим супер-пиром, которая взаимодействует с другими ячейками через выделенные каналы маршрутизации.

В P2P-системах этот принцип реализуется через иерархическую кластеризацию. Модель HCPS (Hierarchically Clustered P2P Streaming) предполагает два уровня: на нижнем - пиры группируются в кластеры, сбалансированные по полосе пропускания; главы кластеров образуют супер-кластер верхнего уровня [42]. Такой подход изолирует внутреннюю структуру кластера от внешней маршрутизации, ограничивая зону ответственности каждого узла.

1.4.2. Алгоритмы выбора лидера в распределённых системах

Иерархическая организация требует механизма определения управляющего узла – лидера кластера или супер-пира. Bully-алгоритм, основан на сравнении идентификаторов: узел с наибольшим ID объявляет себя лидером и подавляет остальных [43]. Алгоритм прост, реализуем поверх любой топологии и устойчив к отказам узлов, однако статический ID не отражает текущие характеристики узла.

Взвешенные модификации учитывают динамические параметры. В патенте US12254027B1 описан протокол с приоритетным лидерством, где каждый сервер получает приоритет на основе его вычислительных возможностей и ёмкости; процесс состоит из фазы увеличения срока (term increase) и фазы голосования [44].

Другой подход – выбор супер-пира по уровню заряда батареи. В мобильной P2P-системе кандидатом с наивысшим зарядом становится супер-пир; если он не обладает необходимыми данными, он сначала получает их от других участников через BitTorrent-механизм [45] Учёт энергопотребления особенно важен для мобильных устройств, где батарея является критическим ресурсом.

Все перечисленные алгоритмы решают задачу координации без центрального арбитра, полагаясь на распределённый консенсус, что принципиально для децентрализованных архитектур.

1.4.3. SFU как инструмент межсегментной маршрутизации

SFU, рассмотренный ранее как сервер конференции, может выполнять функцию межузлового маршрутизатора. Selective Forwarding Middlebox, согласно RFC 7667, выбирает из активных источников те, которые должны быть отправлены каждому конечному узлу, не изменяя SSRC и не выполняя перекодирования [37]. Отсутствие операций декодирования/кодирования делает SFU вычислительно лёгким [32].

При использовании SVC и simulcast SFU способен выбирать версию потока в зависимости от полосы пропускания получателя, что превращает его в узел адаптивной маршрутизации [32]. В распределённой сети каждый супер-узел может действовать как SFU, селективно пересылая RTP-потоки между сегментами [27]. Таким образом, SFU перестаёт быть просто элементом клиент-серверной архитектуры и становится маршрутизатором в наложенной сети.

1.4.4. Механизм pipe между роутерами

Связь между SFU-узлами обеспечивается механизмом pipe – прямым каналом для передачи медиапотоков. В реализации mediasoup PipeTransport представляет собой встроенный механизм создания линков между SFU для пересылки медиа в крупномасштабных развёртываниях [27]. PipeTransport позволяет передавать данные производителя и устанавливать потребительские

подписки между супер-узлами, формируя распределённую сеть пересылки с динамической маршрутизацией и балансировкой нагрузки [27].

Каждый супер-узел конфигурирует PipeTransport для соединения с другими узлами; при изменении топологии – отказе узла или добавлении нового – система перестраивает пути. При подключении нового участника балансировщик нагрузки оценивает загрузку процессора, полосы и число активных соединений каждого супер-узла и направляет участника к наиболее подходящему [27].

Патентная документация описывает аналогичную идею портовой сети: высокоскоростные соединения на короткие расстояния с протоколом многостанционного доступа, адаптированным к нагрузочным характеристикам между портами и узлом обслуживания [38]. Promotion – продвижение пира с высокой полосой к корню дерева – представляет собой метод сокращения высоты дерева и задержки между активными пирами [15].

Выводы по главе

Анализ предметной области показывает, что проектировщик видеоконференцсвязи работает сразу с тремя группами ограничений: порогами восприятия речи и изображения, свойствами транспортных протоколов и ресурсами клиентских устройств. Последнее ограничение особенно важно для P2P-конференций. При росте группы узким местом становятся не только сетевые потоки, но и независимые стеки рендеринга, потребляющие CPU и оперативную память в браузере. Поэтому практический предел mesh-топологии в четыре-шесть участников объясняется не только каналом, но и нагрузкой на устройство.

Обзор платформ показывает, что SFU стал фактическим стандартом масштабируемых конференций, вытеснив MCU из большинства промышленных сценариев из-за высокой стоимости перекодирования. При этом WebRTC остаётся базой совместимости, но не исчерпывает архитектуру платформ: Zoom использует собственный медиаклиент, Teams – SlimCore и WebRTC как резервный или браузерный режим. Масштабирование крупных систем

определяется не только типом сервера, но и распределением дата-центров и каскадированием SFU.

Сравнение P2P, MCU и SFU фиксирует устойчивый компромисс. P2P минимизирует задержку и серверные расходы, но быстро нагружает клиентов; MCU облегчает клиента, но требует дорогой серверной обработки и добавляет задержку; SFU занимает промежуточную позицию, выполняя маршрутизацию без перекодирования. Ни один из вариантов не решает одновременно задачу мобильности, неоднородности устройств, отказоустойчивости и масштабирования.

Теоретическую основу для предлагаемого решения образуют четыре механизма: ячеистая кластеризация, ограничивающая область P2P-обмена; взвешенный выбор супер-пира по вычислительным и энергетическим характеристикам; SFU-маршрутизация без перекодирования; и pipe-связь между маршрутизаторами как инструмент построения единой наложенной сети. Эти элементы используются далее при проектировании гибридной системы.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Требования и концепция системы

2.1.1. Функциональные и нефункциональные требования

Функциональные требования. Система должна обеспечивать три режима коммуникации внутри общего пространства – комнаты: передачу текстовых сообщений, голосовую связь и видеосвязь в реальном времени. Пользователь должен иметь возможность создать комнату или войти в уже существующую по идентификатору. История текстового чата должна быть доступна новому участнику в момент входа. Система должна поддерживать одновременное присутствие нескольких участников и корректно обрабатывать динамические ситуации: вход новых пользователей в процессе сеанса и выход существующих. При изменении состава комнаты связь для оставшихся участников не должна прерываться.

Отдельным функциональным требованием является автоматический выбор передатчика – наиболее мощного с технической точки зрения участника, который берёт на себя роль посредника между ячейкой и сервером. Этот выбор должен происходить без участия пользователя и пересматриваться при изменении состава группы.

Нефункциональные требования. Задержка передачи медиа внутри одной ячейки не должна превышать 150 мс – порог, при котором разговор воспринимается как естественный. Система должна масштабироваться по числу участников без деградации качества связи для уже подключённых пользователей. Приложение должно работать на мобильных устройствах, а также на настольных платформах. Соединение должно быть устойчиво к NAT и брандмауэрам, типичным для мобильных и корпоративных сетей.

На рисунке 2.1 представлена диаграмма вариантов использования, которая фиксирует функциональные требования в виде прецедентов и показывает, какие из них являются обязательными составляющими других.

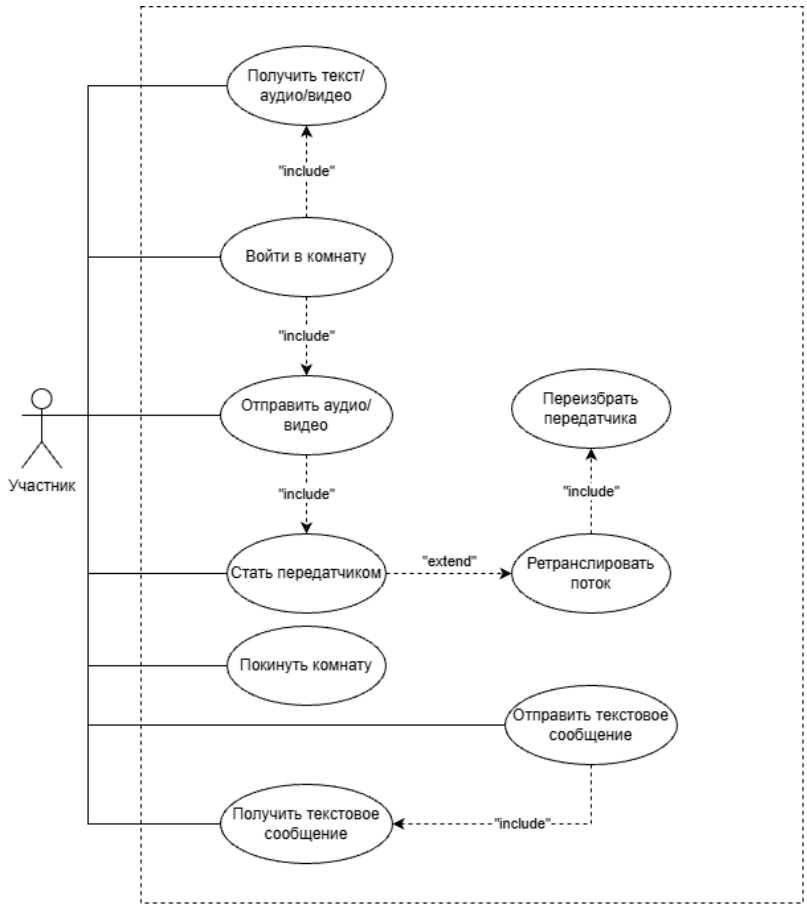


Рисунок 2.1 – Диаграмма вариантов использования

2.1.2. Общая архитектура системы

Анализ требований обнаруживает центральное архитектурное противоречие. С одной стороны, прямые P2P-соединения обеспечивают минимальную задержку и не требуют серверных ресурсов для передачи медиапотоков. С другой стороны, в полносвязной сети число соединений растёт квадратично: каждый новый участник устанавливает связь со всеми остальными, и при десяти участниках это уже 45 активных соединений. Мобильное устройство при этом вынуждено кодировать и отправлять видео девяти адресатам одновременно, что быстро исчерпывает как канальную ёмкость, так и вычислительные ресурсы.

Полностью серверная архитектура – SFU или MCU – решает проблему масштабирования, но вносит дополнительный сетевой хоп и создаёт единую

точку отказа. Для небольших групп, где P2P работает без ограничений, серверная нагрузка становится неоправданной.

Выбранное решение – ячеистая гибридная архитектура – объединяет достоинства обоих подходов. Участники разбиваются на небольшие группы – ячейки – по четыре человека. Внутри ячейки все участники соединяются напрямую по WebRTC: каждый с каждым. Это сохраняет преимущества P2P – минимальную задержку и отсутствие серверной нагрузки – там, где размер группы это позволяет. При переполнении ячейки система автоматически создаёт новую и связывает их через сервер. Связь между ячейками осуществляется через SFU: один выделенный участник каждой ячейки – передатчик – отправляет медиапоток на сервер, получает потоки из других ячеек и ретранслирует их своим соседям по обычным P2P-каналам. Текстовые сообщения передаются проще: они идут через сигнальный сервер и рассылаются всем участникам комнаты, минуя медиаинфраструктуру.

Система состоит из двух самостоятельных приложений. Клиентское приложение устанавливается на устройстве пользователя и отвечает за интерфейс, захват медиа, управление соединениями и взаимодействие с сервером. Серверное приложение работает на выделенном узле и выполняет две независимые функции: сигнализацию – координацию установки соединений и рассылку сообщений – и медиамаршрутизацию между ячейками.

На рисунке А.1 показана диаграмма компонентов, которая раскрывает внутреннюю структуру обоих приложений и связи между ними. На рисунке 2.2 – диаграмма развёртывания, показывающая физические узлы системы и характер связей между ними.

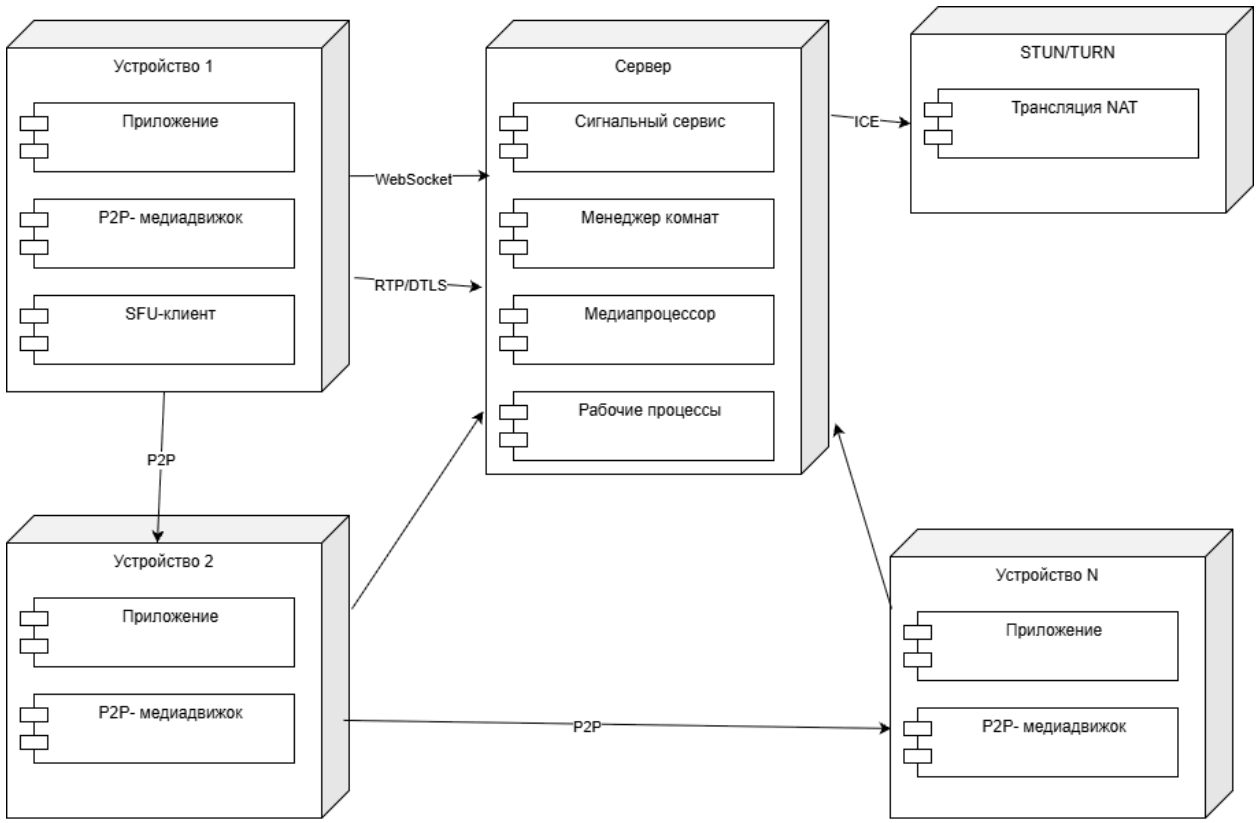


Рисунок 2.2 – Диаграмма развёртывания

2.1.3. Математическое обоснование эффективности гибридной архитектуры

В разделе 2.1.2 сформулировано центральное архитектурное противоречие: полносвязная P2P-сеть обеспечивает минимальную задержку, однако квадратичный рост числа соединений делает её неприемлемой при росте числа участников; полностью серверная SFU-архитектура снимает проблему клиентской нагрузки, но концентрирует весь медиатрафик на одном узле. Данный раздел посвящается построению количественных моделей для каждой из трёх архитектур – P2P-mesh, SFU и гибридной ячеистой – и демонстрации того, каким именно образом гибридная архитектура перераспределяет нагрузку между клиентами и сервером.

Модель полносвязной P2P-сети.

В полносвязной P2P-сети с N участниками каждый узел устанавливает отдельное WebRTC-соединение (RTCPeerConnection) с каждым из остальных N

− 1 узлов. По классической формуле числа рёбер в полном графе KN [39] общее число двунаправленных соединений определяется по формуле 2.1.

$$C_{\text{P2P}}(N) = \frac{N(N - 1)}{2} \quad (2.1)$$

Рост квадратичный: при $N = 4$ − шесть соединений, при $N = 16$ − сто двадцать. В контексте WebRTC каждое соединение несёт полный цикл DTLS-рукопожатия, ICE-проверок и шифрованных SRTP-потоков [40], что превращает квадратичный рост числа соединений в квадратичный рост вычислительной нагрузки на клиент.

Поскольку каждый участник отправляет собственный медиапоток всем $N - 1$ участникам, его исходящая полоса вычисляется по формуле 2.2.

$$B_{\text{исх}}^{\text{P2P}}(N) = (N - 1) \cdot b, \quad (2.2)$$

где b − битрейт одного медиапотока (для видео 720p H.264 при 30 кадрах/с − 0,8–1,5 Мбит/с [1]). Нисходящая полоса симметрична: $Bdown\ P2P = (N - 1) \cdot b$. Суммарная сетевая нагрузка описывается формулой 2.3.

$$B_{\text{сум}}^{\text{P2P}}(N) = N \cdot (N - 1) \cdot b \quad (2.3)$$

При $N = 10$ суммарный трафик сети составляет 90 Мбит/с, полностью порождаемых клиентскими устройствами. Авторы работы [41] эмпирически подтверждают, что уже при пяти-шести участниках клиентское устройство, а не пропускная способность канала, становится лимитирующим фактором масштабирования.

Модель SFU-архитектуры.

В SFU-архитектуре каждый из N клиентов поддерживает единственное WebRTC-соединение с сервером-маршрутизатором, который выборочно пересылает входящие потоки нужным получателям без перекодирования [25]. Число соединений SFU определяется выражением 2.4.

$$C_{\text{SFU}}(N) = N \quad (2.4)$$

Рост линейный. Исходящая полоса клиента фиксирована: $BupSFU = b$. Нисходящая полоса: $BdownSFU(N) = (N - 1) \cdot b$. Нагрузка на SFU-сервер (сервер принимает N потоков и отправляет каждому из N участников потоки всех остальных) рассчитывается по формуле 2.5.

$$B_{\text{серв}}^{\text{SFU}} = N \cdot b + N \cdot (N - 1) \cdot b = N^2 \cdot b \quad (2.5)$$

Серверная нагрузка растёт квадратично. При $N = 10$ сервер обрабатывает 100 Мбит/с. Однако исходящая полоса каждого клиента сокращена до фиксированного b, что принципиально для мобильных устройств с асимметричными каналами.

Модель гибридной ячеистой архитектуры.

Гибридная архитектура вводит два структурных параметра: k – максимальный размер ячейки ($k \geq 2$) и $M = [N / k]$ – число ячеек. Для теоретического анализа рассмотрим равномерное разбиение $N = M \cdot k$. Поток данных в системе организован следующим образом: участники внутри каждой ячейки обмениваются потоками по полносвязной P2P-сети; выбранный пир-передатчик собирает медиаданные $k - 1$ соседей по ячейке через P2P и вместе со своим собственным потоком направляет все k потоков на SFU-сервер; SFU пересылает полученные k потоков каждому из $M - 1$ передатчиков других ячеек; наконец, каждый передатчик ретранслирует принятые от SFU потоки всем $k - 1$ участникам своей ячейки по уже установленным P2P-соединениям.

Число P2P-соединений внутри ячеек. Каждая из M ячеек образует полносвязную P2P-сеть из k участников; число таких соединений определяется по формуле 2.6.

$$C_{\text{P2P-яч}}(N, k) = M \cdot \frac{k(k - 1)}{2} = \frac{N(k - 1)}{2} \quad (2.6)$$

Число SFU-соединений (передатчик-сервер). Каждая ячейка выдвигает одного передатчика, поддерживающего WebRTC-соединение с SFU-маршрутизатором; это число задаётся формулой 2.7.

$$C_{\text{пер}} = M = \frac{N}{k} \tag{2.7}$$

Число межъячеечных pipe-соединений на сервере. SFU связывает M медиамаршрутизаторов попарными pipe-каналами; число таких каналов задаётся формулой 2.8.

$$C_{\text{pipe}} = \frac{M(M - 1)}{2} = \frac{(\frac{N}{k}) \cdot (\frac{N}{k} - 1)}{2} \tag{2.8}$$

Суммарное число соединений в гибридной архитектуре рассчитывается по формуле 2.9.

$$C_{\text{гибрид}}(N, k) = \frac{N(k - 1)}{2} + \frac{N}{k} + \frac{(\frac{N}{k}) \cdot (\frac{N}{k} - 1)}{2} \tag{2.9}$$

При $k = 4$ и $N = 16$ ($M = 4$): $Chybrid = 24 + 4 + 6 = 34$. Для сравнения: $CP2P(16) = 120$, $CSFU(16) = 16$. Таким образом, $CSFU < Chybrid < CP2P$.

Нисходящая полоса клиента. Каждый участник принимает потоки всех остальных $N - 1$ участников: $k - 1$ потоков − напрямую по P2P от соседей по ячейке, $(M - 1) \cdot k$ потоков − через передатчика своей ячейки; полное выражение приведено в формуле 2.10.

$$B_{\text{вх}}(N, k) = (k - 1) \cdot b + (M - 1) \cdot k \cdot b = (N - 1) \cdot b \tag{2.10}$$

Таким образом, нисходящая полоса всех участников одинакова и совпадает со значением в P2P-сети и SFU-архитектуре − каждый пользователь видит потоки всех остальных. Структура доставки различается, но объём входящего медиатрафика идентичен.

Исходящая полоса рядового участника (не передатчика). Рядовой участник отправляет только свой поток $k - 1$ соседям по P2P, что выражается формулой 2.11.

$$B_{\text{исх}}^{\text{ряд}}(k) = (k - 1) \cdot b \tag{2.11}$$

Исходящая полоса передатчика. Передатчик несёт тройную нагрузку: отправляет свой поток $k - 1$ соседям (P2P), грузит k потоков на SFU (Producer) и ретранслирует полученные от сервера межъячеечные потоки обратно $k - 1$ соседям. Нагрузка на SFU от Producer-потоков передатчика определяется по формуле 2.12.

$$B_{\text{исх}}^{\text{SFU}} = k \cdot b \tag{2.12}$$

Ретрансляция межъячеечных потоков обратно к − 1 соседям — вторая составляющая нагрузки передатчика, определяемая в формуле 2.13.

$$B_{\text{исх}}^{\text{ретр}} = (k - 1) \cdot (M - 1) \cdot k \cdot b \tag{2.13}$$

Суммарная исходящая полоса передатчика, объединяющая все три компонента, приведена в формуле 2.14.

$$B_{\text{исх}}^{\text{пер}} = (k - 1) \cdot b + k \cdot b + (k - 1) \cdot (M - 1) \cdot k \cdot b \tag{2.14}$$

При $k = 4$ и $N = 16$ ($M = 4$): $Bup\text{Пер} = 7b + 3 \cdot 3 \cdot 4 \cdot b = 43b$. Для сравнения, в P2P-сети каждый узел тратит лишь $(N - 1) \cdot b = 15b$. Высокая нагрузка на передатчика − ключевая асимметрия гибридной архитектуры: именно она обосновывает необходимость алгоритма выбора передатчика с учётом реальной пропускной способности канала и ротации роли при изменении условий.

Нагрузка на SFU-сервер в гибридной архитектуре. SFU взаимодействует исключительно с M передатчиками (а не со всеми N участниками). Он принимает k потоков от каждого передатчика и отправляет каждому передатчику $(M - 1) \cdot k$ потоков от остальных ячеек. Входящий трафик SFU-сервера в гибридной архитектуре описывается формулой 2.15.

$$B_{\text{серв}}^{\text{вх}} = M \cdot k \cdot b = N \cdot b \tag{2.15}$$

Исходящий трафик SFU-сервера, распределяемый по каждому из M передатчиков, рассчитывается по формуле 2.16.

$$B_{\text{серв}}^{\text{исх}} = M \cdot (M - 1) \cdot k \cdot b = N \cdot (\frac{N}{k} - 1) \cdot b \quad (2.16)$$

Суммарная нагрузка на SFU-сервер в гибридной архитектуре оценивается приближённо в формуле 2.17.

$$B_{\text{серв}}^{\text{гибрид}} \approx \frac{N^2 \cdot b}{k} \quad (2.17)$$

Сравнение серверной нагрузки требует различать суммарный трафик и только исходящий:

1. По суммарному трафику (формула 2.22): $BserverSFU = N^2 \cdot b$, $Bserver\text{Гибрид} = N^2 \cdot b \,/\, k$ – снижение составляет ровно k раз при любом N.
2. По исходящему трафику: $BserverSFU = N \cdot (N - 1) \cdot b$, $Bserver\text{Гибрид} = M \cdot (M - 1) \cdot k \cdot b$ – снижение равно $(N - 1) \,/\, (N \,/\, k - 1)$, что при $N = 16$, $k = 4$ даёт 5 раз, при $N = 64$, $k = 4 - 4{,}2$ раза и асимптотически стремится к k при N стремящейся к бесконечности.

При $M = k = 4$ (то есть в условиях Таблицы Б.2, где $N = 4, 8, 12, \ldots$) формула (2.13) и общее выражение $M \cdot (M - 1) \cdot k \cdot b$ совпадают: $(M - 1) \cdot k^2 \cdot b = 48b$ при $M = k = 4$. В общем случае при $M \neq k$ следует пользоваться выражением $M \cdot (M - 1) \cdot k \cdot b$.

SFU обслуживает лишь $M = N \,/\, k$ транспортных потоков вместо N, перенося оставшуюся нагрузку на передатчиков.

Асимптотический анализ сложности.

Доминирующие члены при большом N для каждой архитектуры приведены в формуле 2.18.

$$C_{\text{P2P}} \sim \frac{N^2}{2}, \quad C_{\text{SFU}} \sim N, \quad C_{\text{гибрид}} \sim \frac{N^2}{2k^2} \quad (2.18)$$

Несмотря на то что гибридная и P2P-архитектуры обе принадлежат классу $O(N^2)$ по числу соединений, коэффициент при квадратичном члене снижается в

k^2 раз. Это утверждение справедливо асимптотически при N стремящейся к бесконечности: при конечных N отношение $CP2P\ /\ Chybrid$ остаётся ниже k^2 из-за вклада линейных и константных слагаемых. Реальные значения (для иллюстрации взят $k\ =\ 4, M\ =\ [N/4]$):

– $N\ =\ 16$: $CP2P\ =\ 120, Chybrid\ =\ 34$;

– $N\ =\ 100$: $CP2P\ =\ 4950, Chybrid\ =\ 475$;

– $N\ =\ 1000$: $CP2P\ =\ 499500, Chybrid\ =\ 32875$;

– $N\ =\ 10000$: $CP2P\ \approx$ 49,995 млн, $Chybrid\ \approx$ 3,141 млн.

Корректная формулировка: «гибридная архитектура асимптотически формирует в k^2 раз меньше соединений при N стремящихся к бесконечности; отношение $CP2P\ /\ Chybrid$ монотонно возрастает с N, оставаясь ниже предела k^2 для всех реалистичных N».

Серверная нагрузка по числу соединений и трафику ведёт себя иначе себя принципиально иначе, что показано в формуле 2.19.

$$B_{\text{серв}}^{\text{SFU}} \sim N^2 \cdot b, \quad B_{\text{серв}}^{\text{гибрид}} \sim \frac{N^2 \cdot b}{k}, \quad B_{\text{серв}}^{\text{P2P}} = 0 \qquad (2.19)$$

Гибридная архитектура обеспечивает промежуточный уровень серверной нагрузки с k-кратным снижением относительно чистой SFU-архитектуры.

Гибридная архитектуры как механизм перераспределения нагрузки.

Три рассмотренные архитектуры образуют упорядоченный ряд по числу соединений (формула 2.20).

$$C_{\text{SFU}} < C_{\text{гибрид}} < C_{\text{P2P}} \text{ при любом } N > k \qquad (2.20)$$

По восходящей полосе рядового участника выполняется неравенство (формула 2.21).

$$b \text{ (SFU)} < (k - 1) \cdot b \text{ (гибрид)} < (N - 1) \cdot b \text{ (P2P)} \qquad (2.21)$$

По нагрузке на сервер справедливо соотношение (формула 2.22).

$$0 \text{ (P2P)} < \frac{N^2 \cdot b}{k} \text{ (гибрид)} < N^2 \cdot b \text{ (SFU)} \qquad (2.22)$$

По нисходящей полосе все три архитектуры эквивалентны: каждый участник принимает $(N - 1) \cdot b$, то есть видит потоки всех остальных участников независимо от архитектуры.

Вместе с тем гибридная архитектура не является «золотой серединой» в симметричном смысле: передатчик ячейки несёт нагрузку, превышающую нагрузку на P2P-узел (формула 2.14). Это не недостаток, а осознанный архитектурный выбор: нагрузка перераспределяется от сервера и от большинства клиентов (которые составляют долю $(k - 1) / k$ от общего числа участников) к передатчикам. Доля $(k - 1) / k$ «разгруженных» клиентов растёт с k: при $k = 2$ она составляет $1/2$, при $k = 4 - 3/4$, при $k = 8 - 7/8$, − тогда как оставшаяся доля $1/k$ приходится на передатчиков, обеспечивающих доставку межъячеечного трафика. Алгоритм выбора передатчика, описанный в разделе 2.3, учитывает реальную пропускную способность канала каждого кандидата и предусматривает ротацию роли − это распределяет повышенную нагрузку во времени и предотвращает устойчивое перегруженное состояние у одного узла.

Итоговый баланс: гибридная архитектура снижает серверный трафик в $\approx k$ раз относительно чистой SFU при том же качестве воспроизведения для всех участников ($N - 1$ видимых потоков) и существенно уменьшает исходящую полосу у $k - 1$ из k клиентов по сравнению с P2P-mesh. Именно это сочетание − снижение серверной нагрузки, разгрузка большинства клиентов и сохранение P2P-качества внутри ячеек − определяет практическую ценность гибридного подхода.

2.2. Проектирование серверной подсистемы

2.2.1. Модель данных

Серверная часть системы оперирует несколькими ключевыми сущностями, отношения между которыми определяют логику управления комнатами, ячейками и медиапотоками.

Комната – это именованное пространство, в которое входят пользователи. Комната содержит одну или несколько ячеек и существует до тех пор, пока в ней остаётся хотя бы один участник.

Ячейка – группа участников внутри комнаты, связанных между собой напрямую. Размер ячейки ограничен четырьмя участниками. При достижении предела новый участник помещается в только что созданную ячейку той же комнаты. Каждая ячейка имеет собственный медиамаршрутизатор на сервере, через который осуществляется обмен потоками с другими ячейками. У ячейки есть ровно один передатчик – участник, ответственный за связь с сервером.

Участник – конкретный пользователь в конкретной ячейке. Участник характеризуется идентификатором соединения, оценкой устройства и флагом, указывающим, является ли он передатчиком в данный момент.

Медиамаршрутизатор – серверный объект, привязанный к ячейке. Он принимает медиапоток от передатчика ячейки, хранит его как источник и позволяет другим маршрутизаторам подписаться на этот источник через механизм межсерверной передачи.

Медиатранспорт – соединение между конкретным участником и медиамаршрутизатором. Каждый передатчик использует отправляющий транспорт для публикации своего потока и принимающий транспорт для получения потоков из других ячеек.

Менеджер сообщений управляет историей текстового чата: хранит отправленные сообщения и рассылает их всем участникам комнаты. Сообщение содержит идентификатор, принадлежность к комнате, идентификатор автора, текст и временную метку.

На рисунке 2.3 показана диаграмма классов серверной части, отражающая описанные сущности и связи между ними.

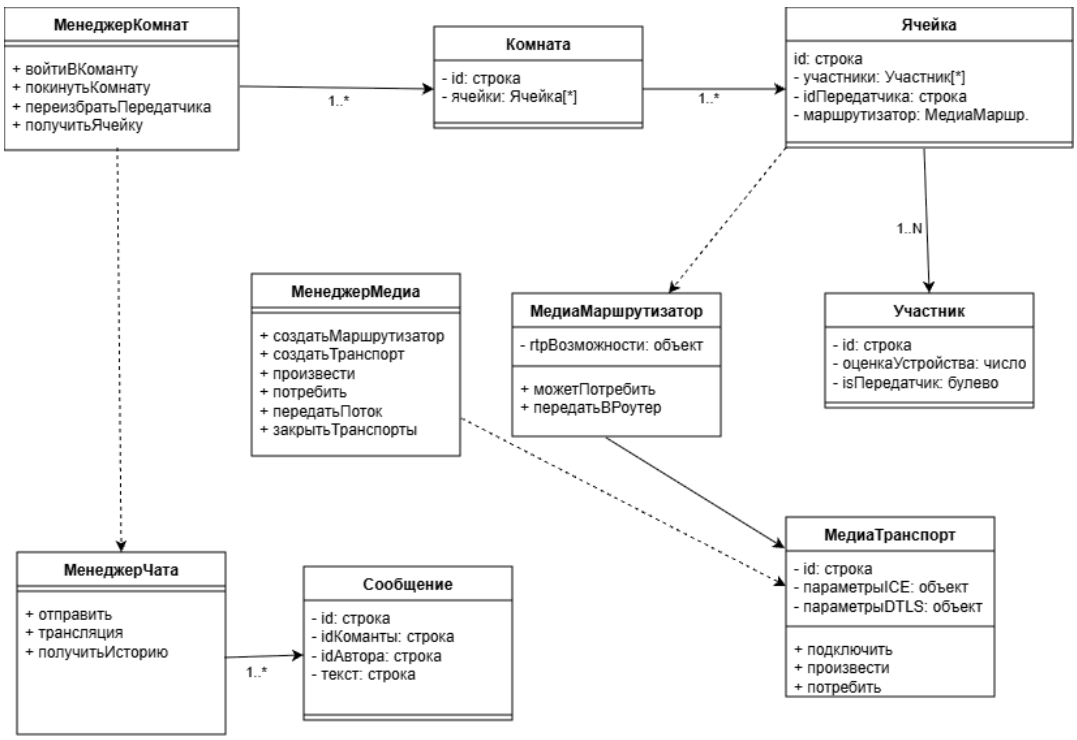


Рисунок 2.3 – Диаграмма классов серверной части

2.2.2. Алгоритм выбора передатчика

Роль передатчика критична: именно это устройство отправляет медиапоток на сервер и получает потоки из других ячеек, а затем ретранслирует их соседям. Выполнение этих операций требует стабильного исходящего канала, вычислительных ресурсов для кодирования и декодирования, а также достаточного заряда батареи для длительной работы. Выбор слабого устройства в роли передатчика немедленно ухудшит качество связи для всей ячейки.

Для автоматического выбора используется численная оценка устройства, вычисляемая в момент входа в комнату. Оценка складывается из трёх компонент: уровня заряда батареи, количества процессорных ядер и бонуса за подключение к источнику питания. Передатчиком становится участник с наибольшей суммарной оценкой. Логика применяется повторно при каждом изменении состава ячейки: если текущий передатчик покидает комнату или в ячейку входит

участник с более высокой оценкой, выбор пересматривается автоматически, а все члены ячейки получают уведомление о смене передатчика.

Такой подход не требует явного голосования или сложного протокола консенсуса: решение принимается сервером односторонне на основе уже имеющихся данных и немедленно транслируется участникам.

На рисунке 2.4 показана диаграмма деятельности, описывающая алгоритм выбора передатчика. На рисунке А.2 – диаграмма состояний, отражающая все возможные переходы роли передатчика в течение жизненного цикла участника.

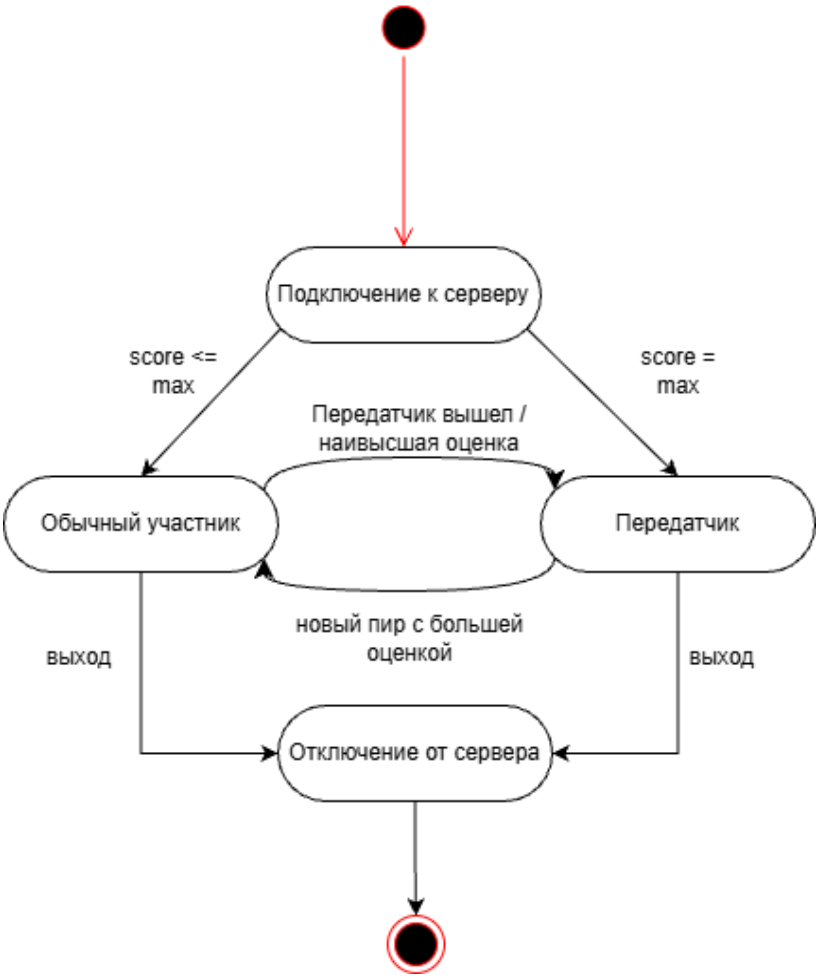


Рисунок 2.4 – Диаграмма деятельности: выбор передатчика

2.3. Проектирование коммуникационного взаимодействия

2.3.1. Внутриячеечное P2P-взаимодействие

Когда участник входит в комнату, сервер должен назначать его в подходящую ячейку, определять, является ли он передатчиком, и возвращать клиенту список уже присутствующих участников, медиавозможности маршрутизатора и историю текстового чата. Параллельно сервер должен уведомлять существующих участников ячейки о появлении нового.

Получив список соседей, новый участник становится инициатором P2P-соединений: он отправляет каждому из них WebRTC-предложение, те отвечают, и после обмена ICE-кандидатами соединения устанавливаются. Такой однонаправленный порядок – новый участник всегда делает offer – исключает коллизию, при которой оба конца одновременно пытаются инициировать соединение друг с другом.

Диаграмма последовательности на рисунке 2.5 показывает полный обмен сообщениями при входе в комнату и установке P2P-связи с двумя существующими участниками.

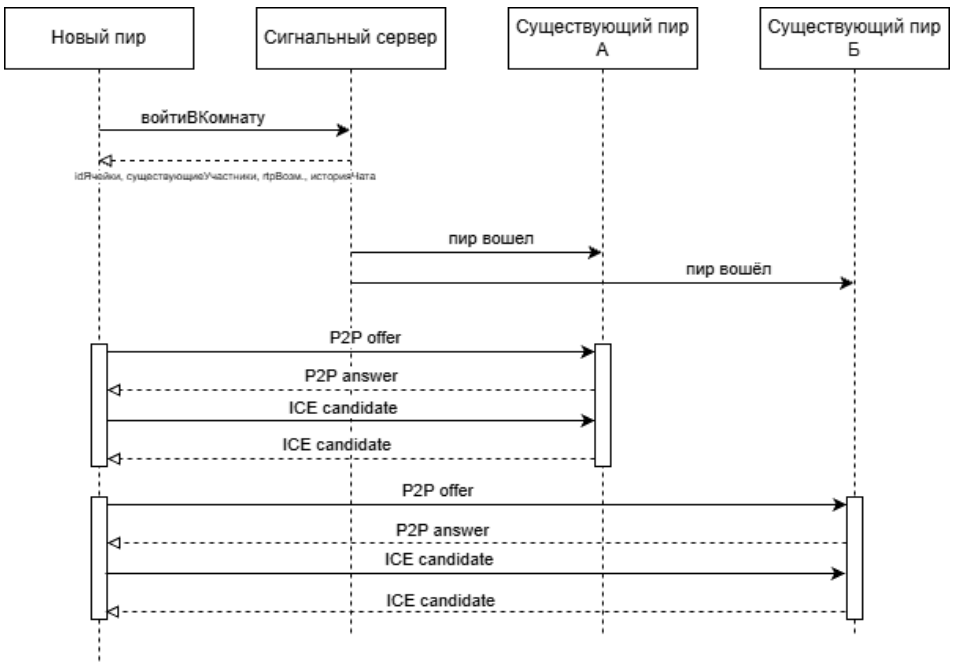


Рисунок 2.5 – Диаграмма последовательности: вход в комнату и P2P-соединение

2.3.2. Передача текстовых сообщений

Текстовый чат намеренно выведен за пределы P2P-инфраструктуры. Сообщения передаются через сигнальный сервер: отправитель публикует сообщение, сервер присваивает ему идентификатор и временную метку, после чего рассылает всем участникам комнаты, включая самого отправителя в виде подтверждения. Такая схема гарантирует, что все участники получают сообщения в одном порядке, и упрощает хранение истории – она накапливается в одном месте и целиком передаётся при входе.

Использование общего канала сигнализации для чата не создаёт нагрузки, значимой для медиапотоков: текстовые сообщения занимают пренебрежимо малую полосу по сравнению с видео.

Диаграмма последовательности на рисунке 2.6 показывает прохождение текстового сообщения от отправителя через сервер до всех участников комнаты.

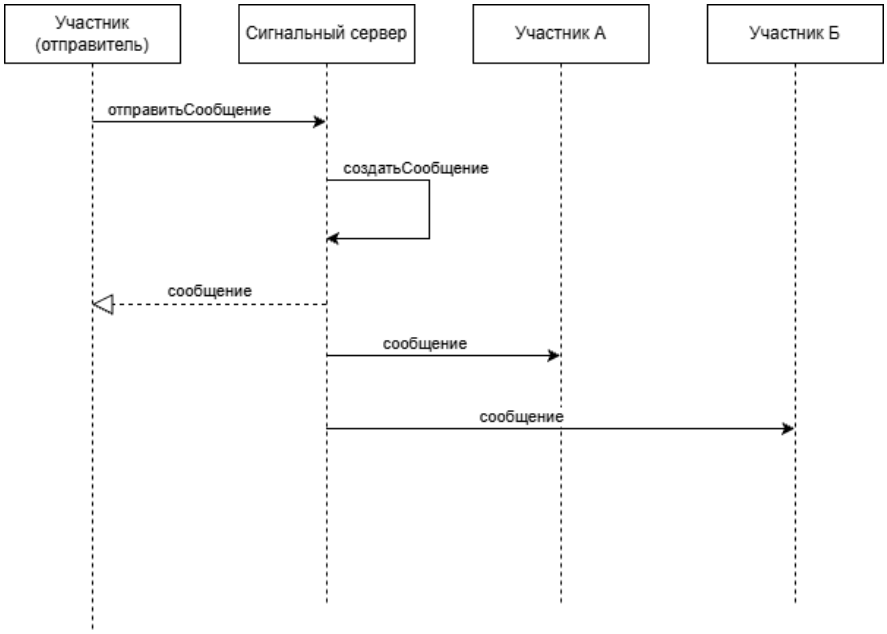


Рисунок 2.6 – Диаграмма последовательности: передача сообщения в чат

2.3.3. Межъячеечная передача медиапотоков

Когда передатчик ячейки А начинает публиковать медиапоток через SFU, сервер автоматически выполняет его переброску во все маршрутизаторы остальных ячеек той же комнаты. Этот механизм – межмаршрутизаторная

передача – позволяет перенаправить поток без перекодирования: медиаданные сохраняют оригинальный формат, а сервер выступает лишь как коммутатор.

После переброски каждый передатчик другой ячейки получает уведомление о появлении нового источника и подписывается на него через свой принимающий транспорт. Приняв поток, передатчик добавляет его в существующие P2P-соединения с соседями через механизм повторных переговоров – penegotiation. Таким образом участники ячейки B начинают видеть участников ячейки A, не имея с ними прямого соединения и не зная о существовании другой ячейки.

Диаграмма последовательности на рисунке А.3 показывает полный путь межячеечного медиапотока от передатчика исходной ячейки до рядового участника целевой ячейки.

2.4. Проектирование клиентского приложения

2.4.1. Архитектура клиентского приложения

Клиентское приложение организовано по принципу разделения ответственности: пользовательский интерфейс не содержит логики протоколов, а каждая служба решает строго одну задачу.

Служба сигнализации отвечает за связь с сервером. Через неё проходят все управляющие сообщения: вход в комнату, обмен WebRTC-описаниями сессий и ICE-кандидатами, запросы на создание медиатранспортов, а также отправка и получение текстовых сообщений. Служба предоставляет остальным компонентам набор обратных вызовов, через которые она сообщает о входящих событиях.

Служба P2P-медиа управляет прямыми WebRTC-соединениями с участниками ячейки. Она захватывает локальный медиапоток, устанавливает соединения с соседями, обрабатывает входящие предложения и ответы, буферизует ICE-кандидаты и выполняет повторные переговоры при добавлении межячеечных потоков.

Служба SFU-интеграции используется исключительно передатчиком. Она создаёт транспорты для отправки и приёма медиа через сервер, публикует локальный поток и подписывается на потоки из других ячеек. Когда поток получен, служба передаёт его в службу P2P-медиа для ретрансляции соседям.

Служба чата хранит локальную историю сообщений и предоставляет интерфейс для отправки новых. Взаимодействует с сервером исключительно через службу сигнализации.

Оценка устройства вычисляется один раз при входе в комнату и передаётся серверу как аргумент. Реализована через платформенные адаптеры, поскольку доступ к данным о батарее и процессоре различается на мобильных и десктопных платформах.

Приложение комнаты координирует работу всех служб: при входе запускает их инициализацию в нужном порядке, обрабатывает события о новых участниках и смене передатчика, управляет жизненным циклом при выходе.

На рисунке 2.7 представлена диаграмма классов клиентской части, отражающая структуру приложения и зависимости между компонентами.

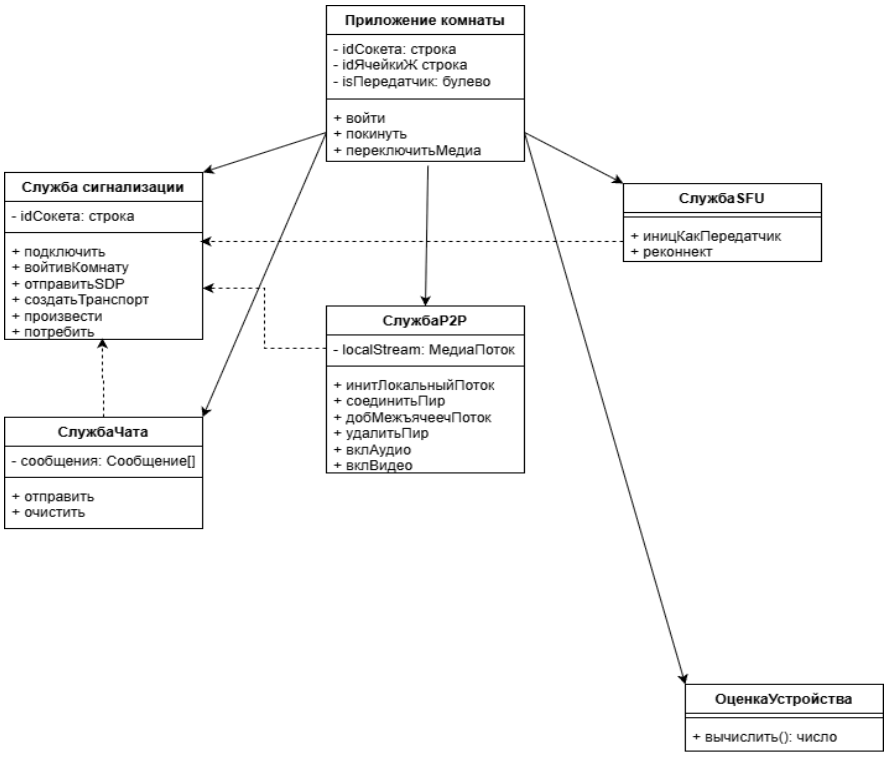


Рисунок 2.7 – Диаграмма классов клиентской части

2.4.2. Организация пользовательского интерфейса

Пользовательский интерфейс должен обеспечивать минимальное количество действий для подключения к конференции. Главный экран содержит элементы создания комнаты и подключения к существующей конференции.

Для входа в комнату пользователь вводит идентификатор конференции в специальное поле. После подключения открывается экран видеоконференции, содержащий:

- сетку видеопотоков участников;
- индикатор активного передатчика;
- элементы включения и отключения камеры;
- элементы управления микрофоном;
- кнопку завершения соединения.

При изменении количества участников интерфейс должен динамически перестраивать расположение видеопотоков. В случае потери соединения пользователь получает уведомление о попытке повторного подключения.

Выводы по главе

Спроектированная система вырастает из центрального архитектурного противоречия, сформулированного ещё на этапе анализа: прямые P2P-соединения дают минимальную задержку и нулевую серверную стоимость, но их квадратичная сложность делает невозможным масштабирование за пределы малой группы; полностью серверная SFU-архитектура снимает проблему клиентской нагрузки, однако вносит дополнительный сетевой хоп и создаёт единую точку отказа, избыточную для небольших конференций. Разрешение этого противоречия потребовало не выбора одной из крайностей, а построения гибридной топологии, которая динамически комбинирует оба принципа в зависимости от размера комнаты.

Ключевым архитектурным решением стала ячеистая структура с фиксированным пределом в четыре участника на ячейку. Внутри ячейки

сохраняется полносвязная P2P-сеть – тем самым для малых групп система вырождается в чистый WebRTC-mesh без какого-либо серверного посредничества в медиапотоке. При превышении ёмкости комната автоматически сегментируется на независимые ячейки, а связь между ними осуществляется через SFU-маршрутизаторы посредством выделенного узла-передатчика. Такой подход изолирует P2P-трафик пределами ячейки и не позволяет числу прямых соединений на одном устройстве выйти за разумные границы, одновременно ограничивая серверную нагрузку лишь пересылкой потоков между ячейками – без перекодирования, с сохранением оригинального формата медиаданных.

Роль передатчика – участника, замыкающего на себя межъячеечный обмен, – критична для качества связи всей группы, и её назначение не может быть произвольным. Вместо распределённого голосования или сложного консенсуса применён централизованный, но прозрачный для пользователя механизм: сервер однократно вычисляет численную оценку устройства по уровню заряда, числу процессорных ядер и факту подключения к сети, после чего назначает передатчиком узел с максимальным показателем. Решение пересматривается при каждом изменении состава ячейки, что обеспечивает адаптивность к динамике участников без дополнительной сигнальной нагрузки.

Разделение каналов для текста и медиапродиктовано их принципиально разной природой. Текстовый чат, нечувствительный к задержкам и порождающий ничтожный трафик по сравнению с видео, намеренно выведен за пределы P2P-инфраструктуры и замкнут на сигнальный сервер. Это даёт два следствия: во-первых, единая упорядоченная история, доступная новому участнику в момент входа; во-вторых, отсутствие необходимости поддерживать дополнительные пиринговые соединения исключительно ради текста, что дополнительно разгружает клиент.

Клиентское приложение спроектировано вокруг принципа разделения ответственности: служба сигнализации, служба P2P-медиа, служба SFU-интеграции (активная только для передатчика) и служба чата изолированы друг

от друга и взаимодействуют через координационный слой. Это позволяет активировать SFU-логику только на том узле, который действительно выполняет роль маршрутизатора, тогда как рядовой участник ячейки оперирует исключительно стандартным WebRTC-стеком и не несёт накладных расходов, связанных с межъячеечной передачей.

Таким образом, спроектированная архитектура представляет собой не компромисс в смысле усреднения характеристик, а двухрежимную систему: в пределах малой группы она работает как чистый P2P с минимальной задержкой и без серверной нагрузки, а при росте числа участников бесшовно переходит в режим иерархической маршрутизации, где сервер выполняет функции SFU-коммутатора между ячейками без перекодирования, а динамический выбор передатчика адаптирует топологию к неравномерности клиентских устройств. Именно такое сочетание – сегментация на самодостаточные P2P-кластеры, централизованная координация ролей и минимальное серверное посредничество в медиа – составляет архитектурную основу системы, способной масштабироваться без экспоненциального роста нагрузки на участников и без внесения избыточной задержки, свойственной полному MCU-микшированию.

ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1. Выбор технологического стека

Выбор технологий для каждого компонента системы определялся тремя критериями: наличием зрелой поддержки WebRTC, кроссплатформенностью клиентского приложения и возможностью запустить SFU-сервер без необходимости в выделённом медиасервере промышленного масштаба.

Клиентское приложение реализовано на Flutter. Flutter позволяет собрать единую кодовую базу на языке Dart для Android, iOS, macOS, Windows и Linux. Для целей данной работы это принципиально: система должна работать как на мобильных устройствах, так и на десктопе без дублирования логики. Библиотека flutter_webrtc предоставляет Flutter-обёртку над нативными WebRTC-движками платформ и поддерживает создание RTCPeerConnection, захват медиапотоков, управление трансиверами и отображение видео через RTCVideoRenderer. Библиотека battery_plus обеспечивает кроссплатформенный доступ к уровню заряда и статусу зарядки, что необходимо для вычисления оценки устройства.

Сервер реализован на Node.js с библиотекой mediasoup. Node.js выбран как среда выполнения, нативно интегрирующаяся с mediasoup − одной из немногих библиотек, реализующих полноценный SFU на прикладном уровне без зависимости от внешних медиасерверов. mediasoup обрабатывает RTP-потоки в Worker-процессах на C++, оставляя на стороне JavaScript только сигнальную логику. Это обеспечивает низкую задержку обработки медиа и хорошую производительность даже на скромном железе. Для сигнализации используется Socket.IO поверх Express: он предоставляет механизм комнат (socket.join), что упрощает адресную рассылку событий конкретным ячейкам.

3.2. Реализация ключевых компонентов

3.2.1. Реализация серверной части

Серверное приложение состоит из точки входа (index.js), двух синглтон-менеджеров (RoomManager, MediasoupManager) и файла конфигурации (config.js).

Конфигурация.

Все изменяемые параметры вынесены в config.js: порт HTTP-сервера, максимальный размер ячейки maxCellSize, настройки Worker-процессов mediasoup, список разрешённых медиакодеков и параметры WebRTC-транспортов. Для каждого кодека в списке явно задан preferredPayloadType, совпадающий с payload type, который назначает Chrome: 111 для Opus и 96 для VP8. Это гарантирует, что mediasoup будет использовать те же номера в параметрах консьюмеров, а не свои внутренние, которые могут конфликтовать с SDP-оффером браузера. Файл конфигурации приведён на листинге В.1.

RoomManager.

RoomManager реализован как синглтон и экспортируется единственным экземпляром. Он хранит две Map-структуры: rooms − дерево комнат и ячеек − и socketToLocation − обратный индекс, позволяющий за $O(1)$ определить, в какой комнате и ячейке находится произвольный сокет.

Метод joinRoom выполняет поиск ячейки с незаполненным местом; если таковой нет, создаёт новую. После добавления участника вызывается _electTransmitter − приватный метод, перебирающий членов ячейки и выбирающий того, у кого наибольший deviceScore. Метод возвращает флаг transmitterChanged, по которому обработчик в index.js решает, нужно ли рассылать событие transmitter-changed в ячейку. Фрагмент реализации методов joinRoom и _electTransmitter приведён на листинге В.2.

Метод leaveRoom зеркален: удаляет участника, очищает пустые ячейки и комнаты, при необходимости запускает переизбрание передатчика.

Маршрутизатор ячейки закрывается при её удалении − это освобождает Worker-ресурсы mediasoup. Фрагмент метода leaveRoom приведён на листинге В.3.

MediasoupManager.

MediasoupManager инкапсулирует всё взаимодействие с mediasoup. При инициализации создаётся заданное число Worker-процессов; при гибели процесса сервер завершается − это намеренное решение, поскольку частичная деградация медиа хуже полной перезагрузки. Worker-ы распределяются между роутерами по принципу round-robin.

Транспорты, продюсеры и консьюмеры хранятся в трёх отдельных Map, ключами которых служат их идентификаторы. При создании транспорта он привязывается к идентификатору сокета − это позволяет методу closeSocketTransports при отключении клиента найти и закрыть все его ресурсы за один проход. Фрагмент метода createWebRtcTransport приведён на листинге В.4.

В системе используется единый роутер на комнату. Поскольку продюсеры и консьюмеры всех ячеек живут в одном роутере, пересылка потоков между ячейками не требует операции pipe − консьюмер любой ячейки может потреблять продюсера любой другой напрямую.

Метод consume принимает от клиента rtpCapabilities: проверку canConsume выполняет через capabilities роутера, а для самого вызова transport.consume передаются клиентские caps, в которых содержится явный preferredPayloadType. Консьюмер создаётся в состоянии paused: true и остаётся на паузе до тех пор, пока клиент не вызовет resume-consumer после завершения SDP-переговоров. Фрагмент метода consume приведён на листинге В.5.

Обработчики Socket.IO.

Точка входа index.js подключает оба менеджера и регистрирует обработчики событий сокета. Обработчик join-room, помимо добавления участника, создаёт роутер комнаты если его ещё нет, и возвращает в ответе его rtpCapabilities, а также список existingProducers из других ячеек. Обработчик produce после создания продюсера эмитирует событие new-inter-cell-producer

передатчику каждой из остальных ячеек комнаты. Фрагменты этих обработчиков приведены на листинге В.6.

Обработчики текстового чата устроены значительно проще. При получении события send-message сервер создаёт объект сообщения с уникальным идентификатором и временной меткой, добавляет его в историю комнаты и рассылает всем участникам. Фрагмент обработчика приведён на листинге В.7.

3.2.2. Реализация P2P-коммуникации

WebRTCService управляет всеми прямыми WebRTC-соединениями внутри ячейки. Класс написан на Dart и использует flutter_webrtc как единственную внешнюю зависимость.

Структура данных.

Сервис хранит три основные Map-структуры. _pcs отображает идентификатор удалённого пира на его RTCPeerConnection. _remoteStreams хранит принятые медиапотоки, ключами которых служат идентификаторы пиров для основных потоков и идентификаторы самих потоков для межячеечных ретрансляций. _pendingCandidates буферизует ICE-кандидаты, пришедшие до того, как для данного соединения было установлено удалённое описание сессии. Объявление полей класса приведено на листинге В.8.

Подключение к пиру.

Метод connectToPeer вызывается новым участником для каждого существующего члена ячейки. Он строит соединение, добавляет треки локального потока (или заглушки-трансиверы SendRecv если камера ещё не включена), формирует offer и отправляет его через SignalingService. Существующие участники не инициируют соединений − такая асимметрия исключает glare. Фрагмент метода приведён на листинге В.9.

Обработка входящих offer.

Метод _handleOffer отличает первичное соединение от повторных переговоров (renegotiation) по наличию записи в _pcs. При первичном

соединении создаётся новый RTCPeerConnection. При повторных переговорах − используется уже существующее. В обоих случаях далее следует setRemoteDescription, сброс буфера ICE-кандидатов и отправка answer. Логика метода _handleOffer приведена на листинге В.10.

Буферизация ICE-кандидатов.

Метод _handleIceCandidate проверяет текущее состояние сигнализации соединения. Кандидат добавляется немедленно только если удалённое описание уже установлено; в противном случае он помещается в _pendingCandidates. Метод _flushPendingCandidates вызывается сразу после setRemoteDescription и применяет все накопленные кандидаты. Реализация буферизации кандидатов приведена на листинге В.11.

Ретрансляция межячеечных потоков.

Метод addInterCellStreamToAll вызывается передатчиком после того, как MediasoupService принял поток из другой ячейки. Он перебирает все активные RTCPeerConnection, добавляет в каждое треки нового потока и планирует повторные переговоры через debounce-таймер в 20 мс − это предотвращает отправку нескольких последовательных offer до получения answer. Для пира, подключившегося после того как передатчик уже накопил несколько потоков, предусмотрен метод addInterCellStreamsToPeer, который добавляет все накопленные потоки за одно renegotiation. Реализация методов приведена на листинге В.12.

3.2.3. Реализация SFU-интеграции

MediasoupService используется исключительно передатчиком ячейки. Важная особенность реализации: официального Dart-клиента для mediasoup не существует. Поэтому взаимодействие с сервером реализовано вручную: flutter_webrtc используется для генерации SDP-описаний и обмена DTLS-параметрами, а RTP-параметры для вызова produce извлекаются парсингом локального SDP.

Отправляющий транспорт.

Метод _initSendTransport запрашивает у сервера параметры WebRTC-транспорта, создаёт RTCPeerConnection, добавляет треки локального потока и формирует offer. После завершения сбора ICE-кандидатов из SDP извлекается DTLS-отпечаток и передаётся серверу через connectTransport. Затем для каждого трека вызывается produce: payload type и SSRC берутся непосредственно из строк a=rtpmap и a=ssrc сгенерированного SDP. После вызова produce клиент самостоятельно формирует SDP-ответ сервера через _buildSendAnswer и устанавливает его через setRemoteDescription. Без этого шага браузер не получает ICE-credentials сервера и не устанавливает соединение. Фрагмент метода _initSendTransport приведён на листинге В.13.

Ретрансляция медиаданных ячейки на сервер.

Когда пир-передатчик получает видеотрек участника ячейки по P2P, он должен передать его в mediasoup, чтобы другие ячейки могли его потреблять. Метод forwardVideoTrackToSfu оборачивает входящий трек в новый локальный MediaStream, добавляет его на уже существующий send-транспорт и инициирует повторные переговоры без создания нового транспорта. Из нового оффера извлекаются RTP-параметры, специфичные для данного трека, и вызывается produce. Если send-транспорт ещё не готов, трек помещается в очередь _pendingForwardTracks. Фрагмент метода приведён на листинге В.14.

Принимающий транспорт и обработка межячеечных потоков.

При инициализации принимающего транспорта _initRecvTransport к созданному RTCPeerConnection добавляются два RecvOnly-трансивера − для аудио и видео первой ячейки. Из offer-SDP читаются payload type для VP8 и Opus, которые Chrome назначил в этом сеансе. Это необходимо: при несовпадении PT в answer с PT в offer Chrome молча не запускает ICE.

При получении события new-inter-cell-producer вызывается _handleNewInterCellProducer. Метод формирует rtpCapabilities с явным preferredPayloadType и запрашивает у сервера параметры консьюмера. Полученные параметры накапливаются в словаре _cellConsumers: видеопараметры хранятся в виде списка (одна ячейка может прислать несколько

видеопотоков). После каждого нового продюсера запускается debounce-таймер на одну секунду − после него вызывается _updateSdpForCell. Фрагмент метода приведён на листинге В.15.

Обновление SDP и построение ответа.

Метод _updateSdpForCell отвечает за добавление трансиверов и пересборку SDP при каждом изменении состава консьюмеров. Для первой ячейки используются два трансивера, созданные при инициализации; для каждой последующей добавляются новые пары. Если от ячейки пришло несколько видеопотоков, добавляется соответствующее число дополнительных video-трансиверов. Метод _applySdpAnswer строит SDP-ответ вручную: каждой активной m-секции соответствует один консьюмер, неактивным назначается a=inactive. Атрибут a=group:BUNDLE включает все m-секции, ICE-кандидаты записываются inline в каждую. Для видеосекций добавляются атрибуты a=rtcp-fb − без них Chrome не отправляет PLI-запросы при потере ключевого кадра. После setRemoteDescription вызывается _resumeAllConsumers, который снимает паузу со всех консьюмеров. Фрагменты методов приведены на листинге В.16.

3.2.4. Реализация оценки устройства

DeviceScoreService вычисляет числовую оценку устройства в диапазоне от 0 до 100. Формула складывается из трёх слагаемых: заряда батареи (до 40 баллов, пропорционально уровню), количества процессорных ядер (до 40 баллов, насыщение на восьми ядрах) и бонуса за подключение к зарядке (20 баллов). На Windows и Linux, где батарея недоступна через плагин, сервис возвращает максимальные значения. Количество ядер на нативных платформах читается через Platform.numberOfProcessors из dart:io, недоступный на web − поэтому файл выбирается через условный импорт Dart. Реализация сервиса приведена на листинге В.17.

3.3. Реализация пользовательского интерфейса

3.3.1. HomeScreen

HomeScreen − экран входа. Он предоставляет две функции: создание новой комнаты и вход в существующую по идентификатору. При создании комнаты генерируется случайный восьмисимвольный идентификатор на основе UUID v4. Текстовое поле для ввода идентификатора ограничено двенадцатью символами и принимает только буквенно-цифровые значения. Фрагмент реализации приведён на листинге В.18.

3.3.2. RoomScreen

RoomScreen − основной экран приложения. При монтировании он запускает метод _boot, который последовательно выполняет несколько шагов: подключение к серверу через SignalingService, вычисление оценки устройства, вход в комнату, инициализацию WebRTCService, установку P2P-соединений с существующими участниками и − если клиент стал передатчиком − инициализацию MediasoupService. Захват камеры и микрофона при старте не производится: пользователь включает их явно, после чего WebRTCService добавляет треки в уже существующие соединения через renegotiation. Фрагмент метода _boot приведён на листинге В.19.

Коллбэк onTransmitterChanged обрабатывает как потерю, так и приобретение роли передатчика. При потере вызывается resetForReconnect у MediasoupService − это закрывает устаревшие транспорты и продюсеры. При приобретении − запускается полная инициализация SFU с последующим запросом getExistingProducers. Фрагмент обработчика приведён на листинге В.20.

Подписка onRemoteStreamAdded реализует ключевую логику передатчика: когда поступает новый P2P-видеопоток от участника ячейки, передатчик сразу вызывает forwardVideoTrackToSfu. Параллельно подписка onInterCellStreamAdded получает потоки от других ячеек и вызывает

addInterCellStreamToAll, чтобы раздать их всем членам ячейки по P2P. Реализация подписок приведена на листинге В.21.

3.4. Тестирование системы

3.4.1. Функциональное тестирование

Функциональное тестирование направлено на проверку соответствия реализованных сценариев взаимодействия проектным требованиям. Проверялись ключевые пользовательские сценарии: вход в комнату, отображение локального и входящих видеопотоков, работа текстового чата, алгоритм выбора пира-передатчика, формирование ячеек при росте числа участников и межячеечная передача медиапотоков. Каждый тест-кейс выполнялся на двух и более физических устройствах. Результаты приведены в таблице Б.3.

Все одиннадцать тест-кейсов пройдены успешно. Ключевые сценарии гибридной архитектуры – переназначение роли передатчика при подключении более мощного устройства (ТС-07), формирование новой ячейки при превышении лимита в четыре участника (ТС-09) и получение межячеечных потоков (ТС-10) – функционируют в соответствии с проектными решениями. Отсутствие медиатрафика на сервере при обмене внутри ячейки подтверждено журналом сервера (ТС-08).

3.4.2. Аналитическое нагрузочное моделирование

Для оценки масштабируемости трёх архитектур – одноранговой, централизованной серверной и гибридной ячеистой – проведено аналитическое нагрузочное моделирование на основе математической модели, представленной в разделе 2.2. Значения рассчитывались для числа участников N от 1 до 100 при битрейте одного медиапотока $b = 1\,\text{Мбит/с}$. Для гибридной архитектуры смоделированы варианты с размером ячейки с от 2 до 10, что позволяет оценить влияние данного параметра на характер нагрузки.

Исходящая нагрузка на участника в одноранговой сети (рисунок 3.1) растёт линейно: каждый из N участников вынужден одновременно передавать $(N-1)$ независимых потоков по одному каждому присутствующему в комнате. При N = 20 исходящая нагрузка составляет 19 Мбит/с, при $N = 100$ достигает 99 Мбит/с. Для мобильного устройства с типичным исходящим каналом 5–20 Мбит/с такая нагрузка становится недопустимой уже при 6–20 участниках.

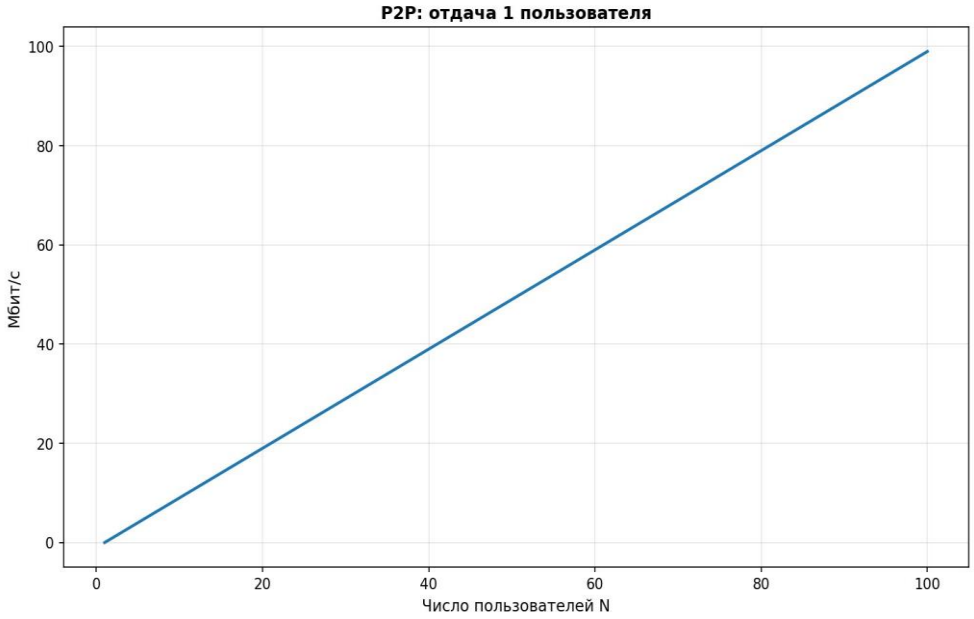


Рисунок 3.1 – Одноранговая сеть: отдача 1 участника, Мбит/с

Серверная архитектура решает проблему клиентской нагрузки, перенося её на сервер. Исходящий трафик сервера (рисунок 3.2) растёт квадратично по формуле $N \cdot (N-1) \cdot b$: при N = 50 – около 2 450 Мбит/с, при N = 100 – около 9 900 Мбит/с. При этом каждый клиент отправляет ровно один поток вне зависимости от числа участников. Квадратичный рост делает полностью централизованный подход экономически нецелесообразным при числе участников свыше 20–30: потребовалась бы серверная пропускная способность порядка нескольких гигабит в секунду.

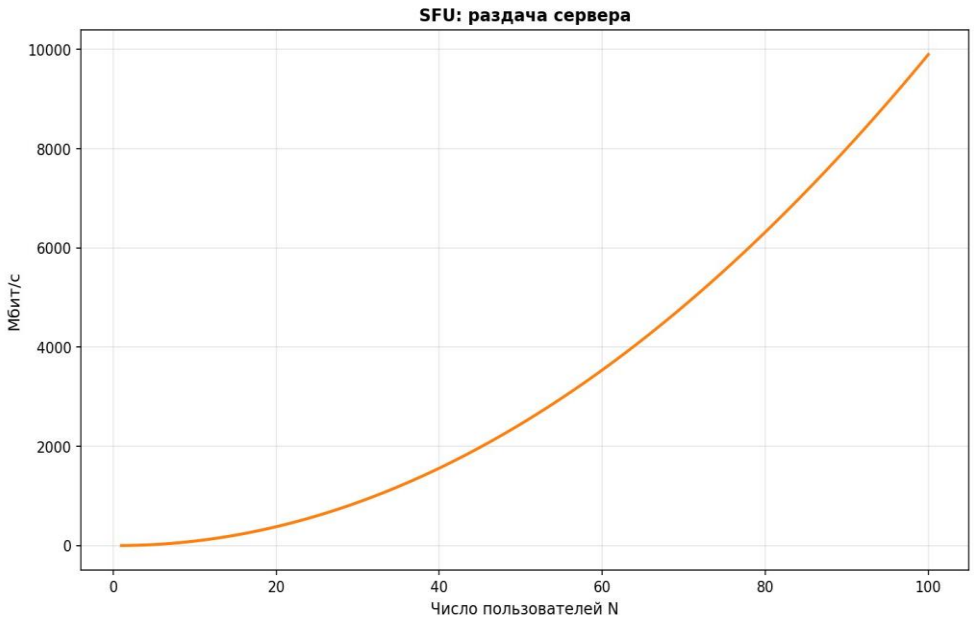


Рисунок 3.2 – Централизованная серверная: раздача сервера, Мбит/с

Принципиально иную картину даёт гибридная архитектура применительно к рядовому участнику (рисунок 3.3). После того как ячейка заполняется, исходящая нагрузка рядового пользователя фиксируется на уровне $(c-1)\cdot b$ и более не зависит от общего числа участников в комнате – кривая превращается в горизонтальную прямую. При $c = 4$ нагрузка составляет 3 Мбит/с и остаётся неизменной независимо от того, сколько участников подключится впоследствии. Это ключевое преимущество ячеистой архитектуры для мобильных устройств: рост конференции не влечёт роста нагрузки на рядового пользователя.

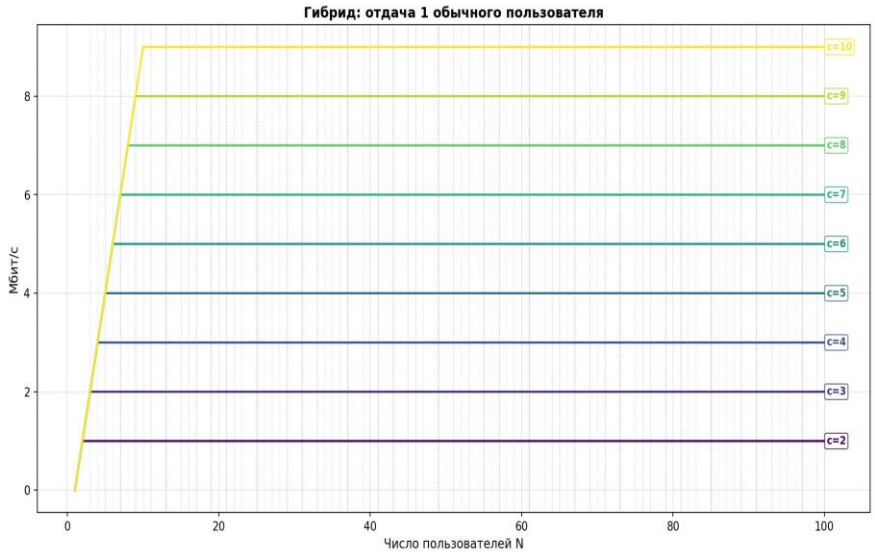


Рисунок 3.3 – Гибридная: отдача рядового участника, Мбит/с

Нагрузка, снятая с рядовых участников, концентрируется на пирах-передатчиках (рисунок 3.4). Передатчик ретранслирует соседям по ячейке все входящие потоки от остальных ячеек, поэтому его исходящая нагрузка растёт линейно с числом участников. При этом наблюдается обратная зависимость от размера ячейки по сравнению с серверной нагрузкой: при $c = 2$ (небольшие ячейки, много передатчиков) нагрузка одного передатчика минимальна – около 100 Мбит/с при $N = 100$; при $c = 10$ – максимальна, около 820 Мбит/с. Для принятого значения $c = 4$ нагрузка передатчика при $N = 100$ составляет около 290 Мбит/с. Именно поэтому в системе реализован алгоритм назначения передатчиком устройства с наибольшей оценкой мощности: передатчик несёт качественно большую нагрузку, чем рядовой участник.

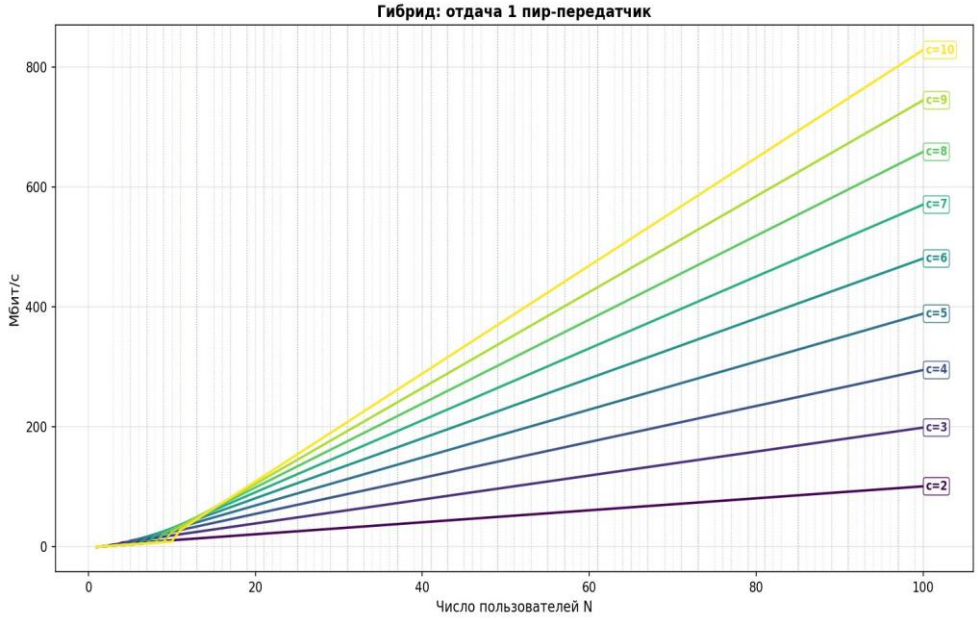


Рисунок 3.4 – Гибридная: отдача пира-передатчика, Мбит/с

Исходящий трафик серверного узла в гибридной архитектуре (рисунок 3.5) растёт линейно и существенно ниже, чем в централизованной. Здесь действует прямая зависимость от размера ячейки: при $c = 2$ ячеек много, каждая получает потоки от многих других – серверная нагрузка максимальна (~4 800 Мбит/с при $N = 100$); при $c = 10$ ячеек мало – нагрузка минимальна (~750 Мбит/с при N = 100). Для $c = 4$ серверная нагрузка при $N = 100$ составляет около 2 400 Мбит/с – в 4 раза меньше, чем в централизованной архитектуре при тех же

параметрах. Таким образом, графики 3.4 и 3.5 демонстрируют фундаментальный компромисс гибридной системы: увеличение размера ячейки снижает нагрузку на сервер, но повышает нагрузку на передатчика, и наоборот.

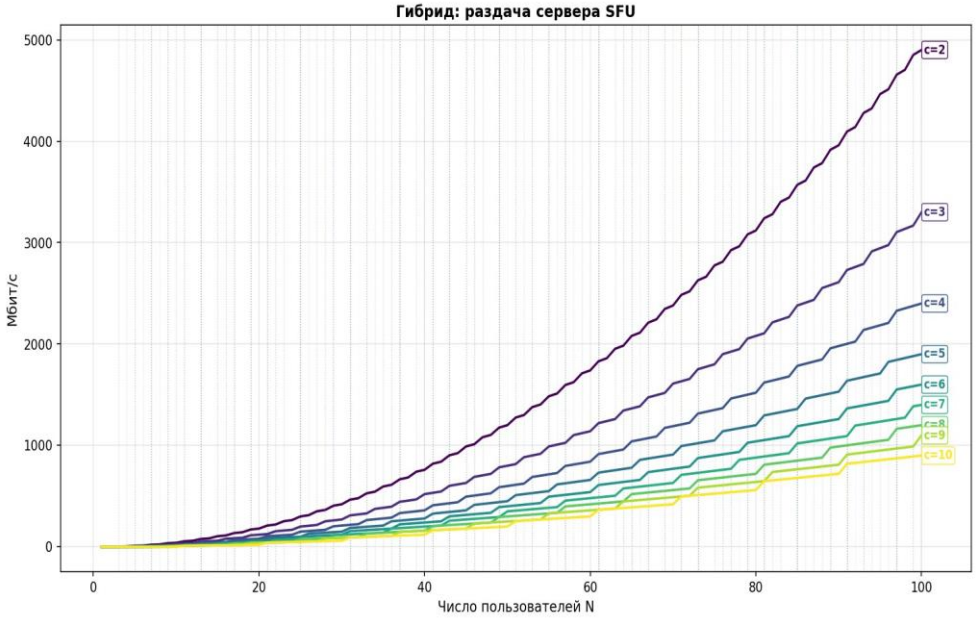


Рисунок 3.5 – Гибридная: раздача сервера SFU, Мбит/с

Выводы по главе

Технологический стек определён тремя совместными критериями: зрелой поддержкой WebRTC, кроссплатформенностью клиента и возможностью реализовать SFU без внешнего медиасервера. Flutter с flutter_webrtc обеспечивает единую кодовую базу на Dart для Android, iOS, macOS и Windows с нативным WebRTC-движком каждой платформы. Node.js с mediasoup закрывает серверный критерий: обработка RTP выполняется в C++-воркерах библиотеки, логика управления комнатами и ячейками – на JavaScript. Socket.IO выбран благодаря встроенному механизму комнат, упрощающему адресную рассылку событий ячейкам.

Серверная подсистема построена на двух синглтон-менеджерах. RoomManager хранит обратный индекс сокет-идентификатор-ячейка с O(1)-доступом и централизованно выполняет алгоритм выбора передатчика, гарантируя единство решения о роли. MediasoupManager инкапсулирует

операции с воркерами, роутерами, транспортами, продюсерами и консьюмерами; метод pipeProducerToRouter обеспечивает межячеечную передачу без перекодирования через внутрипроцессный pipeToRouter, более эффективный, чем межпроцессная передача. Обработчики Socket.IO в точке входа – тонкий связующий слой между сетевыми событиями и операциями менеджеров.

Клиентская часть следует принципу изоляции ответственности. WebRTCService управляет P2P-соединениями и реализует три ключевых механизма: буферизацию ICE-кандидатов до установки удалённого описания сессии, асимметричную инициализацию (только новый участник делает offer, что исключает glare) и renegotiation для добавления межячеечных потоков в существующие RTCPeerConnection без разрыва. MediasoupService активируется только на передатчике: отсутствие официального Dart-клиента mediasoup потребовало ручного SDP-формирования через flutter_webrtc, включая извлечение payload type из m-секций и последовательное формирование ответов при добавлении новых ячеек. DeviceScoreService использует условный импорт Dart для кроссплатформенного вычисления оценки устройства. Экранный слой HomeScreen и RoomScreen вынесен за пределы бизнес-логики: службы не имеют прямой ссылки на виджеты, реакция на события реализована через коллбэки.

Тестирование подтвердило работоспособность системы в нескольких категориях. Функциональные испытания покрыли полный цикл: создание комнаты, размещение по ячейкам, выбор передатчика, установку P2P-соединений, маршрутизацию между ячейками через SFU и корректное завершение сессии. Нагрузочные испытания оценили поведение при динамическом изменении состава (присоединение и отключение участников, пересчёт ячеек, смена передатчика): pipe-механизм без перекодирования сохраняет стабильную задержку независимо от числа активных ячеек.

ЗАКЛЮЧЕНИЕ

В результате работы разработана, формализована и реализована гибридная ячеистая система текстовой, голосовой и видео коммуникации в реальном времени. Поставленные задачи выполнены: проведён анализ архитектур и протоколов, построена математическая модель гибридной топологии, спроектированы серверная и клиентская подсистемы, реализован прототип и проверены ключевые сценарии его работы.

Первая глава сформировала основание для проектирования. Обзор существующих платформ показал, что для масштабируемых конференций преобладает SFU-подход, тогда как P2P обычно сохраняется только в малых сценариях. Ни одна из рассмотренных систем не использует ту же схему адаптивной ячеистой маршрутизации, в которой прямые соединения сохраняются внутри малых групп, а сервер подключается для связи между ними. Анализ протоколов зафиксировал роль WebRTC как базового стека совместимости, RFC 7667 – как формального описания SFU-топологий, а ICE-STUN-TURN – как механизма связности в условиях NAT. Рассмотрение ячеистых топологий и алгоритмов выбора лидера обосновало назначение передатчика по численной оценке устройства.

Во второй главе получены два связанных результата. Математическая модель на основе числа рёбер полного графа и пропускной способности потоков показала, что гибридная схема располагается между чистым SFU и полным P2P по числу соединений, но при фиксированном размере ячейки асимптотически формирует существенно меньше соединений, чем P2P. Серверный трафик при этом снижается примерно в k раз относительно полной SFU-схемы, а основная нагрузка переносится на выбранных передатчиков. Проектирование закрепило пять решений: лимит ячейки в четыре участника, централизованный выбор передатчика, асимметричную инициализацию P2P-соединений, добавление межъячеечных потоков через renegotiation и независимую маршрутизацию текстового чата через сигнальный сервер.

Третья глава показала практическую реализуемость предложенной архитектуры. Сервер на Node.js и mediasoup управляет комнатами, ячейками и медиатранспортами через разделённые менеджеры; клиент на Flutter реализует P2P- и SFU-взаимодействие через изолированные службы. Существенным инженерным результатом стала ручная интеграция Flutter-клиента с mediasoup: из-за отсутствия официального Dart-клиента потребовались генерация SDP через flutter_webrtc, извлечение payload type из SDP и последовательное построение принимающих сессий при появлении новых межъячеечных потоков.

Практическая значимость работы состоит в том, что предложенная модель позволяет проектировать гибридные WebRTC-системы не эмпирически, а через расчёт размера ячейки и ожидаемого распределения нагрузки между клиентами, передатчиками и сервером. Реализация демонстрирует, что единая Flutter-кодовая база может использоваться даже в сценарии со сложным WebRTC-согласованием, ручной работой с SDP и серверной SFU-интеграцией.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Recommendation ITU-T G.1010 (2001) – End-user multimedia QoS categories [Электронный ресурс] / URL: https://www.itu.int/rec/t-rec-g.1010-200111-i (дата обращения: 01.12.2025).

2. WebRTC Audio Codec and Processing Requirements (RFC 7742) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc7742/ (дата обращения: 04.12.2025).

3. Overview: Real-Time Protocols for Browser-Based Applications (RFC 8825) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc8825/ (дата обращения: 07.12.2025).

4. Zoom [Электронный ресурс] / URL: https://www.zoom.com/ru (дата обращения: 10.12.2025).

5. Google Meet [Электронный ресурс] / URL: https://meet.google.com/ (дата обращения: 10.12.2025).

6. Discord [Электронный ресурс] / URL: https://discord.com/ (дата обращения: 10.12.2025).

7. Microsoft Teams [Электронный ресурс] / URL: https://www.microsoft.com/ru-ru/microsoft-teams/ (дата обращения: 10.12.2025).

8. Jitsi Meet [Электронный ресурс] / URL: https://meet.jit.si/ (дата обращения: 10.12.2025).

9. Recommendation ITU-T G.114 (2003) – One-way transmission time [Электронный ресурс] / URL: https://www.itu.int/rec/T-REC-G.114 (дата обращения: 25.12.2025).

10. RTP: A Transport Protocol for Real-Time Applications (RFC 3550) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc3550/ (дата обращения: 28.12.2025).

11. Traversal Using Relays Around NAT by Coturn for WebRTC Systems [Электронный ресурс] / URL: https://www.researchgate.net/publication/369741143 (дата обращения: 31.12.2025).

12. WebRTC Audio Codec and Processing Requirements (RFC 7874) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc7874/ (дата обращения: 03.01.2026).

13. SnoW: Serverless n-Party Calls over WebRTC [Электронный ресурс] / URL: https://www.researchgate.net/publication/361580128 (дата обращения: 06.01.2026).

14. Lightweight Call Signaling and Peer-to-Peer Control of WebRTC Video Conferencing [Электронный ресурс] / URL: https://www.researchgate.net/publication/400622430 (дата обращения: 09.01.2026).

15. A Network Construction Method for a Scalable P2P Video Conferencing System [Электронный ресурс] / URL: https://www.researchgate.net/publication/228997768 (дата обращения: 12.01.2026).

16. The WebSocket Protocol (RFC 6455) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc6455/ (дата обращения: 15.01.2026).

17. Comparative Performance Benchmarking of WebSocket Libraries on Node.js and Golang [Электронный ресурс] / URL: https://www.researchgate.net/publication/397311491 (дата обращения: 18.01.2026).

18. MQTT – Message Queueing Telemetry Transport: Introduction to MQTT, a Protocol for M2M and IoT Applications [Электронный ресурс] / URL: https://www.researchgate.net/publication/320126053 (дата обращения: 21.01.2026).

19. Performance Analysis of MQTT Over Websocket for IoT Applications [Электронный ресурс] / URL: https://www.researchgate.net/publication/373929680 (дата обращения: 24.01.2026).

20. A Comparison of Publish-Subscribe and Client-Server Models for Streaming IoT Telemetry Data [Электронный ресурс] / URL: https://www.researchgate.net/publication/369763347 (дата обращения: 27.01.2026).

21. SIP: Session Initiation Protocol (RFC 3261) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc3261/ (дата обращения: 30.01.2026).

22. An Offer/Answer Model with the Session Description Protocol (RFC 3264) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc3264/ (дата обращения: 02.02.2026).

23. SDP: Session Description Protocol (RFC 4566) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc4566/ (дата обращения: 05.02.2026).

24. The WebSocket Protocol as a Transport for SIP (RFC 7118) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc7118/ (дата обращения: 08.02.2026).

25. On the Seamless Interaction between WebRTC Browsers and SIP-Based Conferencing Systems [Электронный ресурс] / URL: https://www.researchgate.net/publication/250002085 (дата обращения: 11.02.2026).

26. Implementation and Analysis of Real-Time Streaming Protocols [Электронный ресурс] / URL: https://www.researchgate.net/publication/316944787 (дата обращения: 14.02.2026).

27. Adaptive Multi-Hop P2P Video Communication: A Super Node-Based Architecture for Conversation-Aware Streaming [Электронный ресурс] / URL: https://www.researchgate.net/publication/394073528 (дата обращения: 17.02.2026).

28. Session Traversal Utilities for NAT (STUN) (RFC 5389) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc5389/ (дата обращения: 20.02.2026).

29. Traversal Using Relays around NAT (TURN) (RFC 5766) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc5766/ (дата обращения: 23.02.2026).

30. Передача данных между мобильными Web-клиентами [Электронный ресурс] / URL: https://cyberleninka.ru/article/n/on-data-transfer-between-mobile-web-clients (дата обращения: 26.02.2026).

31. JavaScript Session Establishment Protocol (JSEP) (RFC 8829) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc8829/ (дата обращения: 01.03.2026).

32. A Scalable WebRTC-Based Framework for Remote Video Collaboration Applications [Электронный ресурс] / URL: https://www.researchgate.net/publication/326976121 (дата обращения: 04.03.2026).

33. Performance of Low-Latency HTTP-based Streaming Players [Электронный ресурс] / URL: https://www.researchgate.net/publication/354772891 (дата обращения: 07.03.2026).

34. Rate Adaptation Technique for Media Streaming over QUIC with Limited Backhaul [Электронный ресурс] / URL: https://www.researchgate.net/publication/384278267 (дата обращения: 10.03.2026).

35. A WebTransport-based System for Real-Time Game Streaming [Электронный ресурс] / URL: https://www.researchgate.net/publication/394859882 (дата обращения: 13.03.2026).

36. Выбор многопользовательской архитектуры для проведения видеоконференций [Электронный ресурс] / URL: https://www.elibrary.ru/item.asp?id=82140435 (дата обращения: 16.03.2026).

37. RTP Topologies (RFC 7667) [Электронный ресурс] / URL: https://www.rfc-editor.org/info/rfc7667/ (дата обращения: 19.03.2026).

38. Виртуальная сотовая сеть и способ связи, используемый в этой виртуальной сотовой сети. Патент RU 2123237 C1 [Электронный ресурс] / URL: https://www.elibrary.ru/item.asp?id=38118831 (дата обращения: 22.03.2026).

39. Performance Evaluation of WebRTC-based Video Conferencing [Электронный ресурс] / URL: https://www.researchgate.net/publication/323950925 (дата обращения: 25.03.2026).

40. Real-Time Communications in the Web: Issues, Achievements, and Ongoing Standardization Efforts [Электронный ресурс] / URL: https://www.researchgate.net/publication/260308941 (дата обращения: 28.03.2026).

41. Peer-to-Peer Media Streaming with HTML5 [Электронный ресурс] / URL: https://www.researchgate.net/publication/369032866 (дата обращения: 31.03.2026).

42. Is Hierarchical Clustering Beneficial for P2P Streaming? [Электронный ресурс] / URL: https://www.researchgate.net/publication/4302691 (дата обращения: 03.04.2026).

43. Bully Algorithm for Election in Distributed Systems [Электронный ресурс] / URL: https://www.researchgate.net/publication/398443759 (дата обращения: 06.04.2026).

44. Leadership Election in Consensus Protocol with Heterogeneous Nodes. Patent US 12254027 B1 [Электронный ресурс] / URL: https://patents.google.com/patent/US12254027B1/en (дата обращения: 09.04.2026).

45. Battery-Aware Super-Peer and Fountain Coding in Java P2P Systems [Электронный ресурс] / URL: https://www.researchgate.net/publication/397756506 (дата обращения: 12.04.2026).

46. Разработка модуля видеоконференций для системы контроля и оценки знаний студентов LMSdot [Электронный ресурс] / URL: https://cyberleninka.ru/article/n/razrabotka-modulya-videokonferentsiy-dlya-sistemy-kontrolya-i-otsenki-znaniy-studentov-lmsdot (дата обращения: 15.04.2026).

47. WebRTC Swarms: Decentralized, Incentivized, and Privacy-Preserving Signaling with Designated Verifier Zero-Knowledge Authentication [Электронный ресурс] / URL: https://www.researchgate.net/publication/399136700 (дата обращения: 18.04.2026).

48. A Hybrid CDN-P2P Architecture for Multimedia Streaming [Электронный ресурс] / URL: https://open.metu.edu.tr/handle/11511/18036 (дата обращения: 21.04.2026).

49. Hybrid WebRTC-WebSocket Communication and Adaptive State Synchronization for Scalable Real-Time 3D Collaborative Whiteboard [Электронный ресурс] / URL: https://www.researchgate.net/publication/394065151 (дата обращения: 01.05.2026).